

A decorative graphic consisting of three vertical lines of varying heights and colors (yellow, orange, and blue) on the left side of the slide.

2. Pruebas dinámicas de *software*

Pruebas de desarrollo

Técnicas de diseño y herramientas específicas

Miguel Ángel Latre
Javier Nogueras Iso

Índice

☐ Pruebas de unidad

❖ Herramientas

➤ Aserciones de Java

➤ JUnit

☐ Pruebas de integración

❖ Técnicas

➤ Andamiaje

❖ Herramientas

➤ Mockito

PRUEBAS DINÁMICAS DE *SOFTWARE*

Herramientas para la ejecución de pruebas de unidad

Temario (técnicas y herramientas)

	Técnicas	Herramientas
Técnicas generales basadas en la estructura	Puntos de decisión Camino con profundidad	Medición de cobertura
Técnicas generales basadas en la especificación	Particiones equivalencia Pruebas de pares / tablas ortogonales Análisis de valores límite	Pict 3.3 AllPairs
Pruebas de desarrollo	Generales	Aserciones; JUnit 4; JUnit 5/6
de unidad		
de integración	Sustitutos	Mockito
Pruebas de sistema y aceptación (con GUI)	Generales	Selenium Espresso, UI Automator Cucumber
Evaluación (documentación)	Inspecciones Listas de comprobación	
Análisis estático (código)	Patrones de errores Verificación formal	Inspector de IntelliJ/Android Studio JML

Aserciones en Java

❑ Instrucción de Java `assert`

❑ Sintaxis

❖ **`assert`** <condición> [: <expresión>] ;

❑ Semántica

- ❖ Las aserciones pueden estar *activadas* o *desactivadas* a nivel de la máquina virtual
- ❖ Si las aserciones están desactivadas, la instrucción no tiene ningún efecto (la condición y la expresión no son evaluadas)
- ❖ Si las aserciones están activadas, se evalúa la condición
 - Si su resultado es **`true`**, la instrucción no hace nada más
 - Si el resultado es **`false`**, se lanza una `AssertionError`. Si la segunda expresión está presente, se evalúa y su valor se utiliza como argumento del constructor de la `AssertionError`.

Aserciones en Java

```
int x = 8;  
assert x >= 0;  
System.out.println(x);
```

8

```
int x = -1;  
assert x >= 0;  
System.out.println(x);
```

-1

Aserciones
no activadas

Aserciones en Java

```
int x = -1;  
assert x >= 0;  
System.out.println(x);
```

Exception in thread "main"
java.lang.AssertionError
at ...

Aserciones
activadas

```
int x = -1;  
assert x >= 0: "x es negativo";  
System.out.println(x);
```

Exception in thread "main"
java.lang.AssertionError: x
es negativo
at ...

Aserciones activadas,
AssertionError creado con
mensaje

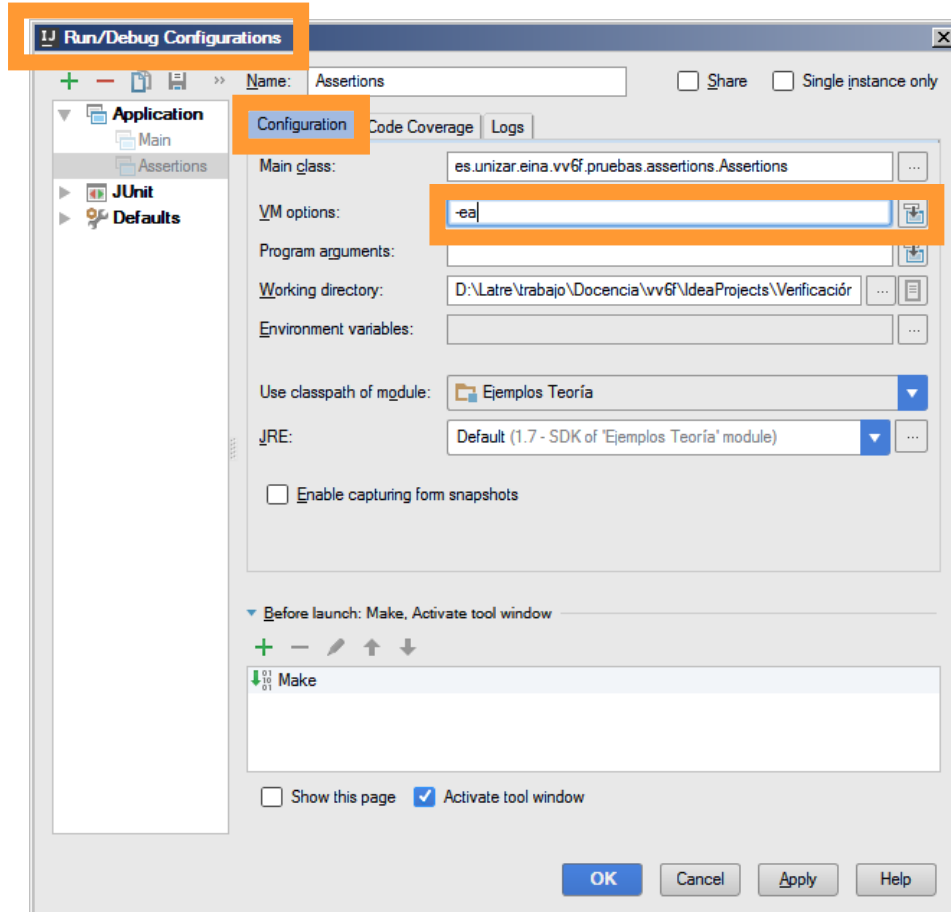
Aserciones en Java

❏ Activación de las aserciones en Java

❖ Al lanzar la máquina virtual

➤ `java -ea Clase`

❖ En IntelliJ:



Aserciones en Java

- ❑ Objetivo de las aserciones:
 - ❖ Evitar la propagación de errores
 - ❖ Localizar la fuente del error, en lugar del síntoma
- ❑ Pensadas para incluirse en el código fuente
- ❑ Mecanismo de activación/desactivación sigue el patrón versión *debug/release* del código fuente.

Entornos de automatización de pruebas

□ *Test harness*

❖ Colección de *software* que facilita la ejecución de pruebas

- Entorno general
- *Scripts* específicos para un programa o proyecto

❖ Objetivos

- Facilitar la realización de juegos de pruebas
- Facilitar la automatización de las mismas
- Generar informes asociados a la ejecución de las pruebas

❖ Beneficios

- Aumento de la productividad debida a la automatización de parte del proceso de pruebas
- Aumento de la probabilidad de que se realicen pruebas de regresión
- Aumento de la calidad del *software* desarrollado

JUnit

❑ JUnit 3

❑ JUnit 4

- ❖ <https://junit.org/junit4/>

- ❖ Integrado en IntelliJ y Android Studio

JUnit4

❑ JUnit 5 / JUnit 6

- ❖ <https://docs.junit.org/>

- ❖ Integrado completamente en IntelliJ

- ❖ Android Studio

- Soporte para pruebas locales

- Con soporte limitado a través de complementos para pruebas instrumentadas

JUnit **5**

 **JUnit**

Juegos de pruebas

- ❑ Un *juego de pruebas* se escribe en una clase Java
- ❑ Una *prueba* se corresponde con un método con la etiqueta `@Test`
 - ❖ Puede haber más de un test en la clase.
 - ❖ Si hay más de un test, deben ser independientes.
 - JUnit no especifica el orden en el que se ejecutarán los test de una clase.
- ❑ El resto de los métodos no definen pruebas.
 - ❖ Pueden ser métodos auxiliares.
 - ❖ Pueden estar etiquetados y tener un significado concreto para JUnit («*test fixtures*»).

Test

□ Estructura habitual de una prueba

- ❖ *Given*: Dada la satisfacción de una precondition
 ➤ Fase de inicialización de la prueba
- ❖ *When*: Cuando se realiza una determinada acción
 ➤ Ejecución del objeto de pruebas
- ❖ *Then*: Entonces se debería satisfacer una postcondición
 ➤ Comprobación de los resultados
 ➤ A través de *aserciones*
 ➤ Idealmente, solo una

Ejemplo de test

```
@Test
public void testAplicarDescuento() {
    // Given: Dado un objeto DescuentoLibros...
    DescuentoLibros descuento = new DescuentoLibros();

    // When: ...cuando se invoca su método aplicarDescuento
    // con 9 libros y 250 €...
    double precio = descuento.aplicarDescuento(9, 250);

    // Then: ... debe obtenerse un precio final de 225 €
    assertEquals(225, precio, 0.001);
}
```

Ejemplo de test

```
@Test
public void testAplicarDescuento() {
    DescuentoLibros descuento = new DescuentoLibros();
    assertEquals(225,
        descuento.aplicarDescuento(9, 250), 0.001);
}
```

Una forma más habitual
de escribir el mismo test

Aserciones

- ❑ Métodos estáticos de la clase `org.junit.Assert` (JUnit4) o de la clase `org.junit.jupiter.api.Assertions` (JUnit5/6):
 - ❖ `fail`
 - ❖ `assertTrue`
 - ❖ `assertFalse`
 - ❖ `assertEquals`
 - ❖ `assertNotEquals`
 - ❖ `assertArrayEquals`
 - ❖ `assertNull`
 - ❖ `assertNotNull`
 - ❖ `assertSame`
 - ❖ `assertNotSame`
 - ❖ `assertThat`

Aserciones

```
@Test
public void testFail() {
    fail();
}
```



```
@Test
public void testFail() {
    fail("Esta prueba siempre falla");
}
```



Texto opcional que se muestra como mensaje de la excepción `AssertionError` que se lanza. Todos los métodos de aserción lo admiten

Aserciones

Expresión que debe evaluarse como **true** (assertTrue) o **false** (assertFalse)

```
@Test
public void testAssertTrue() {
    assertTrue("Fallo: el argumento debería ser true", true);
}

@Test
public void testAssertTrue_fallo() {
    assertTrue("Fallo: 1 ≠ 2", 1 == 2);
}

@Test
public void testAssertFalse() {
    assertFalse("Fallo: ¡1 + 1 = 1!", 1 == 1 + 1);
}

@Test
public void testAssertFalse_fallo() {
    int a = 0;
    assertFalse("Fallo: a debería ser menor que 1", a < 1);
}
```



Texto opcional que se muestra como mensaje de la excepción AssertionError que se lanza.

JUnit 4. Ejemplos

```
@Test  
public void testAssertEquals() {  
    assertEquals("Fallo: las cadenas no son iguales",  
                 "texto", "texto");  
}
```



```
@Test  
public void testAssertEquals_fallo() {  
    assertEquals("Fallo: las cadenas no son iguales",  
                 "esperado", "obtenido");  
}
```



Valor esperado
(normalmente una
constante)

Valor obtenido al
ejecutar el objeto de
pruebas

JUnit 4. Ejemplos

```
@Test  
public void testAssertEquals() {  
    assertEquals("texto", "texto");  
}
```



```
@Test  
public void testAssertEquals_fallo() {  
    assertEquals("esperado", "obtenido");  
}
```



Valor esperado
(normalmente una
constante)

Valor obtenido al
ejecutar el objeto de
pruebas

JUnit 4. Ejemplos

```
@Test  
public void testAssertNotEquals() {  
    int x = 5;  
    assertEquals(4, x);  
}
```



```
@Test  
public void testAssertNotEquals_fallo() {  
    int x = 3;  
    assertEquals(3, x);  
}
```



JUnit 4. Ejemplos

```
@Test
public void testAssertArrayEquals() {
    assertEquals(new int[] {1, 2, 3},
                 new int[] {1, 2, 3});
}
```



```
@Test
public void testAssertArrayEquals_fallo_longitud() {
    assertEquals(new int[] {1, 2, 3},
                 new int[] {1, 2});
}
```



El número de
componentes difiere

```
@Test
public void testAssertArrayEquals_fallo_dato() {
    assertEquals(new int[] {1, 2, 3},
                 new int[] {1, 0, 3});
}
```



El valor de una
componente difiere

JUnit 4. Ejemplos

```
@Test
public void testAssertNull() {
    assertNull(null);
}
```



```
@Test
public void testAssertNull_fallo() {
    assertNull(new Object());
}
```



```
@Test
public void testAssertNotNull() {
    assertNotNull(new Integer(6));
}
```



```
@Test
public void testAssertNotNull_fallo() {
    String str = null;
    assertNotNull(str);
}
```



JUnit 4. Ejemplos

```
@Test
public void testAssertSame() {
    Integer a = new Integer(123);
    Integer b = a;
    assertEquals(a, b);
}
```



```
@Test
public void testAssertSame_fallo() {
    Integer a = new Integer(123);
    Integer b = new Integer(123);
    assertEquals(a, b);
}
```



```
@Test
public void testAssertNotSame() {
    String esperado = "VV";
    String obtenido = new String(esperado);
    assertEquals(esperado, obtenido);
}
```



```
@Test
public void testAssertNotSame_fallo() {
    String esperado = "VV";
    String obtenido = "VV";
    assertEquals(esperado, obtenido);
}
```



«Same» en estas aserciones se refiere a las referencias (punteros) esperados y actuales

Mensajes

❏ Mensajes de error

❖ `assertArrayEquals(FRECUENCIAS_HAMLET,
analizar(HAMLET_FILE));`

➤ Produce el siguiente mensaje de error:
 arrays first differed at element [26];
 expected:<1> but was:<0>

❖ `assertArrayEquals("Hamlet", FRECUENCIAS_HAMLET,
analizar(HAMLET_FILE));`

➤ Produce el siguiente mensaje de error:
Hamlet: arrays first differed at element [26];
 expected:<1> but was:<0>

Nombre de los test



```
@DisplayName("A special test case")
class DisplayNameDemo {

    @Test
    @DisplayName("Custom test name containing spaces")
    void testWithDisplayNameContainingSpaces() {}

    @Test
    @DisplayName("ゝ °□ ° ゝ ")
    void testWithDisplayNameContainingSpecialChars() {}

    @Test
    @DisplayName("🤪")
    void testWithDisplayNameContainingEmoji() {}
}
```

JUnit

❑ Aserciones de la biblioteca Hamcrest

❖ `assertThat([String message,] T value,
Matcher<T> statement)`

❖ Ejemplos:

- `assertThat(x, is(3));`
- `assertThat(x, is(not(4)));`
- `assertThat(response,
 either(containsString("color"))
 .or(containsString("colour")));`
- `assertThat(myList, hasItem("3"));`

❑ <https://hamcrest.org/JavaHamcrest/>

❑ <https://code.google.com/p/hamcrest/wiki/Tutorial>

❑ <https://www.vogella.com/tutorials/Hamcrest/article.html>

Fuente: junit-team. *Matchers and assertThat*. <https://github.com/junit-team/junit4/wiki/Matchers-and-assertthat>

[accedido el 4-3-2026]

JUnit

```
assertTrue(response.contains("color")  
           || response.contains("colour"));  
// ==> failure message:  
// java.lang.AssertionError:
```

Alternativa sin
Hamcrest

```
assertThat(response, anyOf(containsString("color"),  
                           containsString("colour")));  
  
// ==> failure message:  
// java.lang.AssertionError:  
// Expected: (a string containing "color" or a  
//           string containing "colour")  
// got: "Please choose a font"
```

Con Hamcrest

JUnit 4. *Test fixtures*

- ❑ **@Before:** métodos que se ejecutan antes de la ejecución de cada uno de los métodos de prueba.
- ❑ **@After:** métodos que se ejecutan después de la ejecución de cada uno de los métodos de prueba.
- ❑ **@BeforeClass:** métodos estáticos que se ejecutan una única vez antes de cualquier método de prueba de la clase (y antes de cualquier método @Before de la primera prueba).
- ❑ **@AfterClass:** métodos estáticos que se ejecutan una única vez tras la ejecución de todos los métodos de prueba (y después de todos los métodos @After de la última prueba).

- ❑ **@BeforeEach:** métodos que se ejecutan antes de la ejecución de cada uno de los métodos de prueba.
- ❑ **@AfterEach:** métodos que se ejecutan después de la ejecución de cada uno de los métodos de prueba.
- ❑ **@BeforeAll:** métodos estáticos que se ejecutan una única vez antes de cualquier método de prueba de la clase (y antes de cualquier método @BeforeEach de la primera prueba).
- ❑ **@AfterAll:** métodos estáticos que se ejecutan una única vez tras la ejecución de todos los métodos de prueba (y después de todos los métodos @AfterEach de la última prueba).

JUnit 4

❑ Tiempo límite para una prueba

- ❖

```
@Test(timeout = 1000 /* ms */)
public void testWithTimeout() { ... }
```

❑ Tiempo límite para todas las pruebas

- ❖

```
@Rule
public Timeout limiteGlobal
    = new Timeout(10_000 /* ms */);
```

- ❖

```
@Test
public void testBucleInfinito1() {
    while (true) {
    }
}
```

- ❖

```
@Test
public void testBucleInfinito2() {
    while (true) {
    }
}
```

Tiempos límite en JUnit 5



```
@Test
void timeoutNotExceeded() {
    // The following assertion succeeds.
    assertTimeout(ofMinutes(2), () -> {
        // Perform task that takes less than 2 minutes.
    });
}

@Test
void timeoutNotExceededWithResult() {
    // The following assertion succeeds, and returns the supplied
    // object.
    String actualResult = assertTimeout(ofMinutes(2), () -> {
        return "a result";
    });
    assertEquals("a result", actualResult);
}
```


JUnit 4. Deshabilitar test

```
@Ignore("Test is ignored as a demonstration")
@Test
public void testSane() {
    assertThat(1, is(1));
}
```

@Ignore es una alternativa preferible a la de comentar la etiqueta @Test.

Puede ser muy útil en el caso de tests que no son regresión (ej: los tests exploratorios sobre caracteres válidos en el título de una nota).

JUnit 5. Deshabilitar test



```
@Disable("Test is ignored as a demonstration")
@Test
public void testSane() {
    assertThat(1, is(1));
}
```

JUnit 4. Excepciones

- ❑ Resultado esperado de una prueba es una excepción:

```
@Test(expected = IndexOutOfBoundsException.class)
public void empty() {
    new ArrayList<Object>().get(0);
}
```

- ❑ Si se necesita ser más específico:

```
@Test
public void testExceptionMessage() {
    try {
        new ArrayList<Object>().get(0);
        fail("Expected an Exception"); ←
    } catch (IndexOutOfBoundsException ex) {
        assertEquals("Index: 0, Size: 0", ex.getMessage());
    }
}
```

JUnit 5. Excepciones



```
@Test
void exceptionTesting() {
    Throwable exception
        = assertThrows(IllegalArgumentException.class,
            () -> new ArrayList<Object>().get(0));
    assertEquals("Index: 0, Size: 0",
        exception.getMessage());
}
```

Diferencias entre JUnit 3, 4 y 5

	JUnit 3	JUnit 4	JUnit 5
Herencia	junit.framework.TestCase (obligatoria, directa o indirectamente)	Sin restricciones	
Métodos	Prefijo "test"	Etiquetados con @Test	
Métodos accesorios (test fixtures)	--- setUp tearDown ---	@BeforeClass @Before @After @AfterClass	@BeforeAll @BeforeEach @AfterEach @AfterAll
Tiempo límite	Con <i>threads</i>	@Test(timeout = 1000 /*ms*/)	assertTimeout(ofSeconds(1), () -> {});
Excepciones	try {...;fail();} catch (Exception ex) {}	@Test(expected = Exception.class)	assertThrows(Exception.class, () -> {});
Ignorar	/* */	@Ignore	@Disable

Test parametrizados

❑ Secuencia de Fibonacci

Término	Fibonacci
0	0
1	1
2	1
3	2
4	3
5	5
6	8
7	13
8	21

Test parametrizados

Fibonacci

-termino: int
-valorFibonacci: int
«constructor»
+Fibonacci(termino: int)
+valor(): int
-calcular(): int

```
public class Fibonacci {  
  
    private int termino;  
    private int valorFibonacci;  
  
    public Fibonacci(int termino) {  
        this.termino = termino;  
        valorFibonacci = calcular();  
    }  
  
    public int valor() {  
        return valorFibonacci;  
    }  
  
    private int calcular() {  
        if (termino <= 0) { return termino; }  
        else if (termino == 1 || termino == 2) { return 1; }  
        else {  
            return new Fibonacci(n - 1).valor()  
                + new Fibonacci(n - 2).valor();  
        }  
    }  
}
```

¿Test no parametrizados?

```
public class FibonacciTest {

    @Test
    public void test0() {
        assertEquals(0, new Fibonacci(0).valor());
    }

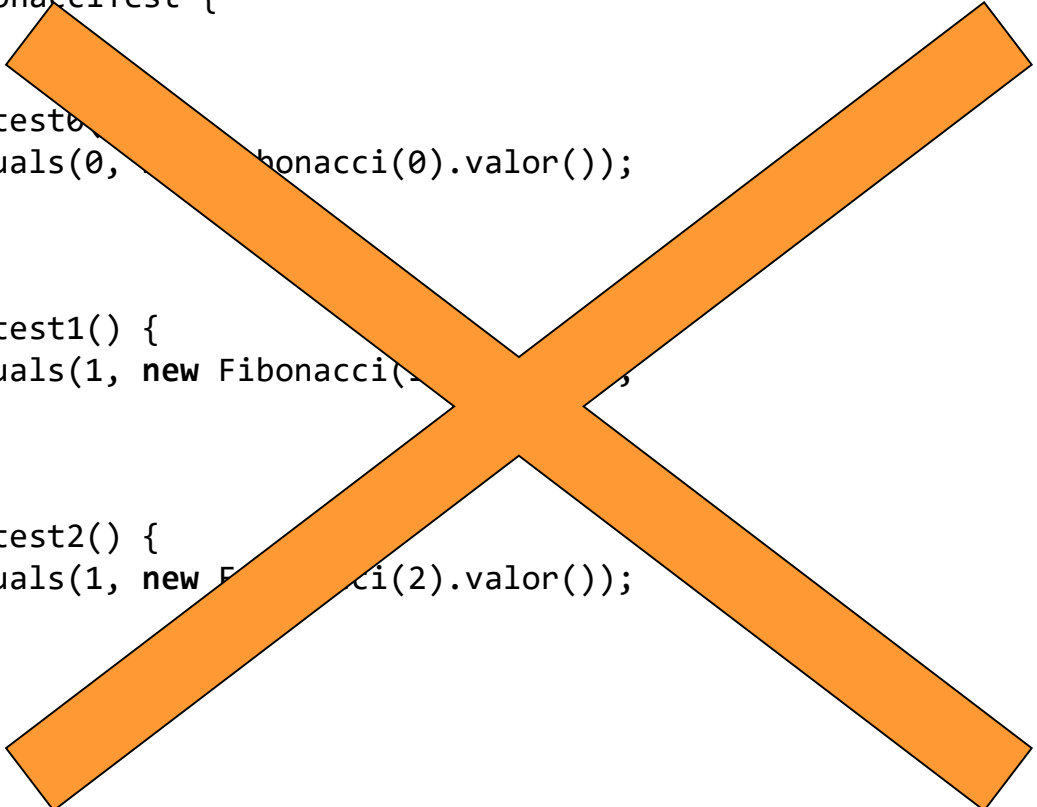
    @Test
    public void test1() {
        assertEquals(1, new Fibonacci(1).valor());
    }

    @Test
    public void test2() {
        assertEquals(1, new Fibonacci(2).valor());
    }

    ...

    @Test
    public void test8() {
        assertEquals(21, new Fibonacci(8).valor());
    }

}
```



JUnit 4. Parametrizado

```
@RunWith(Parameterized.class)
public class FibonacciTest {
    ...
    private int entrada;
    private int esperado;

    public FibonacciTest(int entrada, int esperado) {
        this.entrada = entrada;
        this.esperado = esperado;
    }

    @Test
    public void test() {
        assertEquals(esperado,
                     new Fibonacci(entrada).valor());
    }
}
```

FibonacciTest

–entrada: int

–esperado: int

«constructor»

+FibonacciTest(entrada, esperado: int)

«test» +test(): int

«parameters» +data(): Iterable<Object[]>

JUnit 4. Parametrizado

```
@RunWith(Parameterized.class)
public class FibonacciTest {
```

```
    @Parameters
```

```
    public static Iterable<Object[]> data() {
        List<Object[]> parametros = new ArrayList<>();
        parametros.add(new Object[] { 0, 0 });
        parametros.add(new Object[] { 1, 1 });
        parametros.add(new Object[] { 2, 1 });
        parametros.add(new Object[] { 3, 2 });
        parametros.add(new Object[] { 4, 3 });
        parametros.add(new Object[] { 5, 5 });
        parametros.add(new Object[] { 6, 8 });
        parametros.add(new Object[] { 7, 13 });
        parametros.add(new Object[] { 8, 21 });
        return parametros;
    }
    ...
}
```

FibonacciTest

–entrada: int

–esperado: int

«constructor»

+FibonacciTest(entrada, esperado: int)

«test» +test(): int

«parameters» +data(): Iterable<Object[]>

JUnit 4. Parametrizado

Método @Parameters alternativo

```
@RunWith(Parameterized.class)
```

```
public class FibonacciTest {
```

```
    @Parameters
```

```
    public static Iterable<Object[]> data() {
```

```
        final int[] EXPECTED
```

```
            = new int[] {0, 1, 1, 2, 3, 5, 8, 13, 21};
```

```
        List<Object[]> parametros = new ArrayList<>();
```

```
        for (int i = 0; i < EXPECTED.length; i++) {
```

```
            parametros.add(new Object[] {i, EXPECTED[i]});
```

```
        }
```

```
        return parametros;
```

```
    }
```

```
    ...
```

```
}
```

FibonacciTest

-entrada: int

-esperado: int

«constructor»

+FibonacciTest(entrada, esperado: int)

«test» +test(): int

«parameters» +data(): Iterable<Object[]>

Parametrización en JUnit 5 / 6



```
public class CapicuaTest {  
    @ParameterizedTest  
    @ValueSource(ints = { 0, 1, 11, 121, 3663 })  
    void testCapicuas(int n) {  
        assertTrue(esCapicua(n));  
    }  
  
    @ParameterizedTest  
    @ValueSource(ints = { 10, 12, 123, 5015 })  
    void testNoCapicuas(int n) {  
        assertFalse(esCapicua(n));  
    }  
}
```

Parametrización en JUnit 5 / 6



```
public class FibonacciTest {
    @ParameterizedTest
    @CsvSource({
        "0, 0",
        "1, 1",
        "2, 1",
        "3, 2",
        "4, 3",
        "5, 5",
        "6, 8",
        "7, 13",
        "8, 21",
    })
    public void test(int input, int expected) {
        assertEquals(expected, new Fibonacci(input).valor());
    }
}
```

Parametrización en JUnit 5 / 6

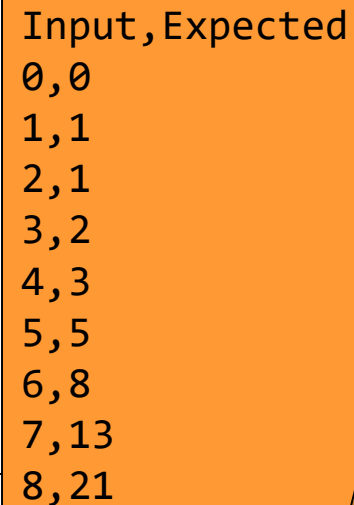


```
public class FibonacciTest {
    @ParameterizedTest(name = "Fibonacci({0}) debería ser {1}")
    @CsvSource({
        "0, 0",
        "1, 1",
        "2, 1",
        "3, 2",
        "4, 3",
        "5, 5",
        "6, 8",
        "7, 13",
        "8, 21",
    })
    public void test(int input, int expected) {
        assertEquals(expected, new Fibonacci(input).valor());
    }
}
```

Parametrización en JUnit 5 / 6



```
public class FibonacciTest {  
    @ParameterizedTest(name = "Fibonacci({0}) debería ser {1}")  
    @CsvFileSource(resources = "/fibonacci-test-cases.csv",  
                    numLinesToSkip = 1)  
    public void testFromFile(int input, int expected) {  
        assertEquals(expected, new Fibonacci(input).valor());  
    }  
}
```

An orange sticky note with a folded bottom-right corner, containing the content of the CSV file.

Input,Expected
0,0
1,1
2,1
3,2
4,3
5,5
6,8
7,13
8,21

fibonacci-test-cases.csv

Test dinámicos en JUnit 5 / 6



```
public class FibonacciTest {

    @TestFactory
    Collection<DynamicTest> generarTestsFibonacci() {
        final int[] EXPECTED = new int[]{0, 1, 1, 2, 3, 5, 8,13,21};
        List<DynamicTest> tests = new ArrayList<>();
        for (int i = 0; i < EXPECTED.length; i++) {
            final int testNumber = i;
            tests.add(dynamicTest("Fibonacci " + testNumber,
                () -> assertEquals(EXPECTED[testNumber],
                                    new Fibonacci(testNumber).valor()))
            ));
        }
        return tests;
    }
}

public static DynamicTest dynamicTest(String displayName, Executable executable)
```


Ejercicio con JUnit.

Secuencia de Collatz más larga

❏ Problema 14 de Project Euler

❖ <https://projecteuler.net/problem=14>

Se define la siguiente secuencia iterativa sobre el dominio de los enteros positivos:

$n \rightarrow n/2$ (si n es par)

$n \rightarrow 3n + 1$ (si n es impar)

Utilizando dicha definición y empezando en 13, se genera la siguiente secuencia:

$13 \rightarrow 40 \rightarrow 20 \rightarrow 10 \rightarrow 5 \rightarrow 16 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 1$

Puede comprobarse que esta secuencia (que comienza en 13 y termina en 1), contiene 10 términos. Pese a que no ha sido probado aún (Conjetura de Collatz), se cree que las secuencias generadas empezando en cualquier número terminan en 1.

¿Qué número inicial, por debajo de un millón, genera la secuencia más larga?

Nota: Una vez la secuencia ha comenzado, es posible que algunos términos superen el límite de un millón.

Ejercicio con JUnit.

Secuencia de Collatz más larga

- ❑ Descargad o clonad el repositorio «unizar-vv-collatz» de GitHub.
 - ❖ <https://github.com/unizar-vv-organization/unizar-vv-collatz>.
- ❑ Hay que configurarlo para utilizar JUnit
- ❑ Observad las clases SecuenciaCollatz, Main y sus métodos.

Ejercicio con JUnit.

Secuencia de Collatz más larga

❑ Clase SecuenciasCollatzTestSiguiente

- ❖ Diseñad pruebas basadas en la especificación para el método `siguienteCollatz` aplicando particiones de equivalencia.
- ❖ Por cada prueba diseñada:
 - Escribid un test en JUnit.
 - Ejecutad el test para comprobar que falla.
 - Añadid el código necesario para que el test pase.
 - Comprobad que la cobertura del método `siguienteCollatz` en modo *tracing* es del 100 %.
- ❖ Usad la secuencia de ejemplo del enunciado como *test basis* para obtener casos de prueba adicionales, aunque sean redundantes.
- ❖ Convertid las pruebas diseñadas en test JUnit parametrizados. Utilizad test parametrizados con las etiquetas `@CsvSource` y `@CsvFileSource` y test dinámicos.

Ejercicio con JUnit.

Secuencia de Collatz más larga

❑ Clase SecuenciasCollatzTestLongitud

- ❖ Diseñad pruebas para el método `longitudCollatz`.
 - Basaos, por una parte, en la secuencia de ejemplo del enunciado. Se trata de una única secuencia, pero podemos distinguir 9 subsecuencias más que terminan en 1.
 - Completad los casos de prueba calculando *a mano* las longitudes de las secuencias iniciadas por los números del 1 al 10.
- ❖ Escribid un test parametrizado en el que ir incluyendo las pruebas diseñadas. Incluir las de una en una y observad como va aumentando la cobertura del código conforme las vais añadiendo.

Ejercicio con JUnit.

Secuencia de Collatz más larga

❑ Clase InicializadorSecuenciaMasLargaTest

- ❖ Diseño de pruebas basadas en la especificación (basados en las longitudes de las secuencias iniciadas por los números del 1 al 10 que habéis calculado antes).

❑ Resto de métodos

- ❖ Añadid pruebas hasta alcanzar una cobertura del 100% en modo *tracing*.

Diseño de las pruebas para el método siguienteCollatz()

Clases de equivalencia

Parámetro	Clases válidas	Clases no válidas
n	$1n > 0$	$4n \leq 0$
	2n par	
	3n impar	

Casos de prueba redundantes según la técnica. Los incluimos porque al estar en el enunciado, los podemos considerar nuestra *test basis*.

Casos de prueba (clases válidas)

#	n	Resultado esperado	Clases cubiertas
P1	40	20	1, 2
P2	13	40	1, 3

P3	20	10	1, 2
P4	10	5	1, 2
P5	5	16	1, 3
P6	16	8	1, 2
P7	8	4	1, 2
P8	4	2	1, 2
P9	2	1	1, 2

Casos de prueba (clases no válidas)

#	n	Resultado esperado	Clases cubiertas
P10	0	Lo obviamos (la comprobación se hace en el constructor)	4

Diseño de las pruebas para el método longitud()

Clases de equivalencia

Parámetro	Clases válidas	Clases no válidas
this.inicio	$1\ n > 0$	$2\ n \leq 0$

Casos de prueba redundantes según la técnica. Los incluimos porque al estar en el enunciado, los podemos considerar nuestra *test basis*.

Casos de prueba (clases válidas)

#	n	Resultado esperado	Clases cubiertas
P1	13	10	1

P2	40	9	1
P3	20	8	1
P4	10	7	1
P5	5	6	1
P6	16	5	1
P7	8	4	1
P8	4	3	1
P9	2	2	1
P10	1	1	1

Casos de prueba (clases no válidas)

#	n	Resultado esperado	Clases cubiertas
P11	0	IllegalArgumentException (al construir el objeto SecuenciaCollatz)	2

Diseño de las pruebas para el método iniciadorSecuenciaMasLarga()

Secuencias de Collatz para establecer una *test basis*

n	Secuencia asociada																			Longitud
1	4	2	1	4	2	1	4	2	1	4	2	1	4	2	1	4	2	1	4	1
2	1	4	2	1	4	2	1	4	2	1	4	2	1	4	2	1	4	2	1	2
3	10	5	16	8	4	2	1	4	2	1	4	2	1	4	2	1	4	2	1	8
4	2	1	4	2	1	4	2	1	4	2	1	4	2	1	4	2	1	4	2	3
5	16	8	4	2	1	4	2	1	4	2	1	4	2	1	4	2	1	4	2	6
6	3	10	5	16	8	4	2	1	4	2	1	4	2	1	4	2	1	4	2	9
7	22	11	34	17	52	26	13	40	20	10	5	16	8	4	2	1	4	2	1	17
8	4	2	1	4	2	1	4	2	1	4	2	1	4	2	1	4	2	1	4	4
9	28	14	7	22	11	34	17	52	26	13	40	20	10	5	16	8	4	2	1	20
10	5	16	8	4	2	1	4	2	1	4	2	1	4	2	1	4	2	1	4	7

Casos de prueba

#	limite	Resultado esperado
P1	2	1
P2	3	2
P3	4	3
P4	5	3
P5	6	3
P6	7	6
P7	8	7
P8	9	7
P9	10	9
P10	11	9

Ejercicio con JUnit.

Secuencia de Collatz más larga

□ Unas soluciones:

❖ <https://github.com/unizar-vv-organization/unizar-vv-collatz-solucion-junit4>

❖ <https://github.com/unizar-vv-organization/unizar-vv-collatz-solucion-junit5>

□ Se recomienda no consultarlas hasta haber intentado previamente su resolución.

PRUEBAS DINÁMICAS DE *SOFTWARE*

Pruebas de integración: andamiaje y herramientas

Temario (técnicas y herramientas)

	Técnicas	Herramientas
Técnicas generales basadas en la estructura	Puntos de decisión Camino con profundidad	Medición de cobertura
Técnicas generales basadas en la especificación	Particiones equivalencia Pruebas de pares / tablas ortogonales Análisis de valores límite	Pict 3.3 AllPairs
Pruebas de desarrollo de unidad	Generales	Aserciones; JUnit 4; JUnit 5
de integración	Sustitutos	Mockito
Pruebas de sistema y aceptación (con GUI)	Generales	Selenium Espresso, UI Automator Cucumber
Evaluación (documentación)	Inspecciones Listas de comprobación	
Análisis estático (código)	Patrones de errores Verificación formal	Inspector de IntelliJ/Android Studio JML

Pruebas de desarrollo

(Recordatorio de Ingeniería del *Software*)

□ Pruebas de unidad

- ❖ Todos los componentes del sistema que se desarrollen se prueban individualmente para comprobar su correcto funcionamiento
- ❖ Se realizan al crear cada módulo o componente individual de un sistema

□ Pruebas de integración

- ❖ Se prueba la integración entre los componentes del sistema para demostrar que se pueden encajar correctamente
- ❖ Son realizadas sobre estos componentes agrupados, se examinan las interfaces para asegurar que estos componentes individuales son llamados cuando es necesario y que los datos que se transmiten entre dichos componentes son los requeridos

Pruebas de desarrollo

(Recordatorio de Ingeniería del *Software*)

- ❑ Problemas de realizar las pruebas de unidad de forma separada
 - ❖ Es necesario crear **módulos auxiliares** que simulen las acciones de los módulos invocados por el módulo que se está probando
 - ❖ Asimismo se han de crear **módulos conductores** para establecer las condiciones necesarias, llamar al módulo objeto de la prueba y examinar los resultados de la prueba

Pruebas de desarrollo

□ Con un entorno de automatización de pruebas:

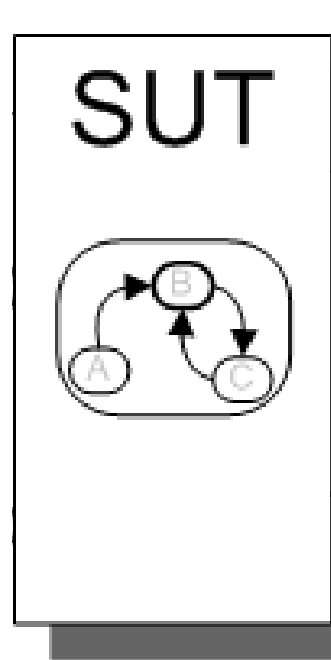
- ❖ Los módulos conductores son los propios test.
- ❖ Los módulos auxiliares pueden ser:
 - Los módulos colaboradores reales o
 - Sustitutos de los módulos reales
 - Andamiaje o *scaffolding*

Andamiaje

- ❑ **Unidad objeto de pruebas:** módulo sobre el que se realizan pruebas (*System Under Test, SUT*)
- ❑ **Colaborador:** módulo del que depende la unidad objeto de pruebas (*Depended-on Component, DOC*)
 - Puede utilizarse el colaborador real
 - Puede no estar implementado
 - Puede que la interacción con él sea muy lenta
 - Puede que la preparación del colaborador para las pruebas sea muy laboriosa
 - Puede que la interacción con el mismo no sea factible
- ❑ **Sustituto o doble:** módulo usado en las pruebas en lugar del colaborador real (*Test Double*)
 - *Dummy*
 - *Fake*
 - *Stub*
 - *Spy*
 - *Mock*
 - *Proxy*

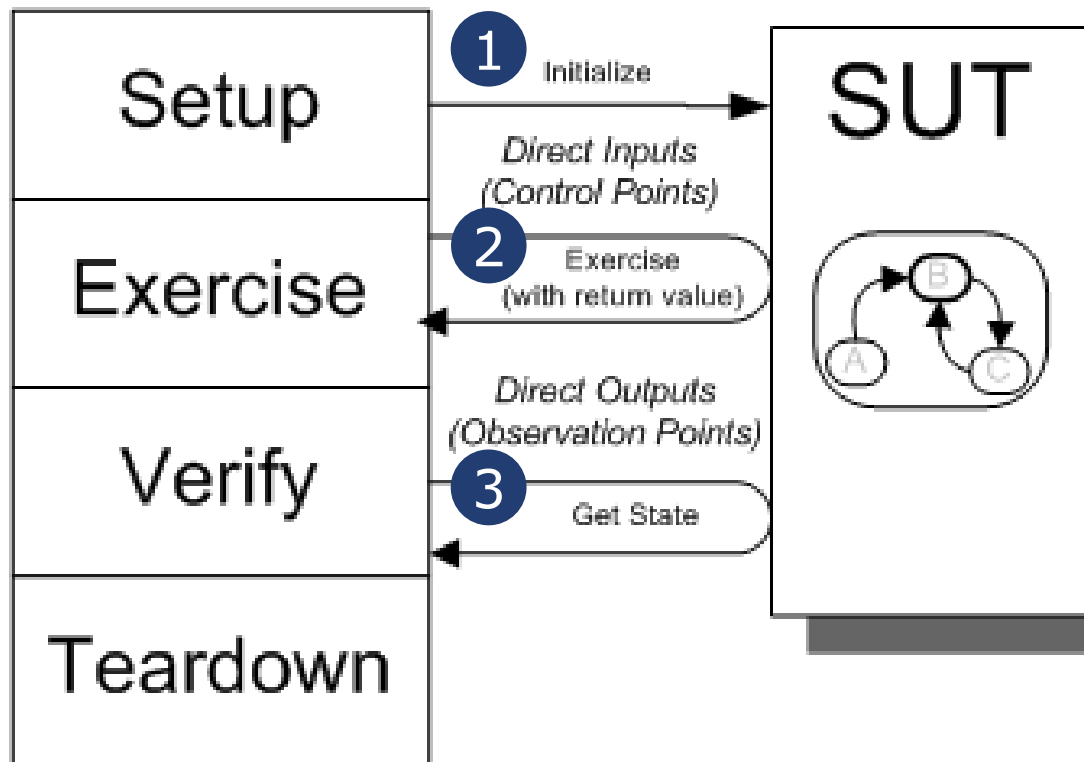
Andamiaje

- Unidad objeto de pruebas (*System under test*)



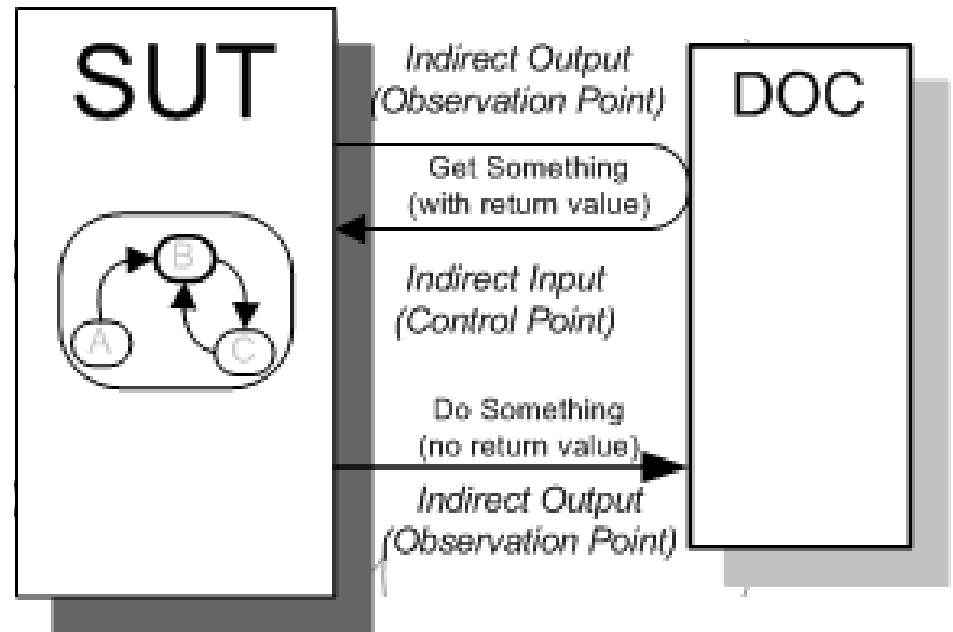
Andamiaje

Esquema test de unidad



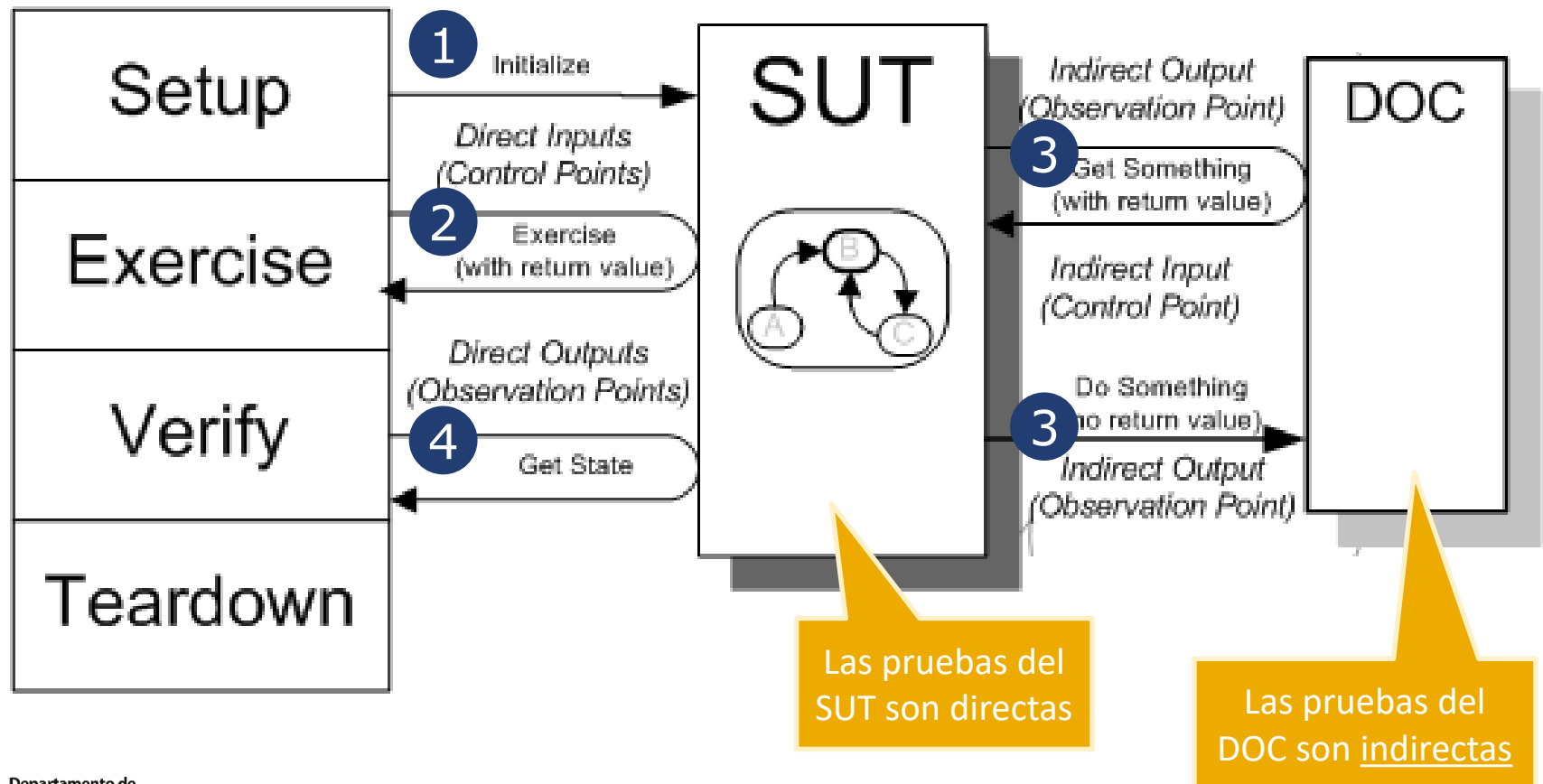
Andamiaje

- Unidad objeto de pruebas con dependencias de otra unidad colaboradora (*Depended-on Component*)



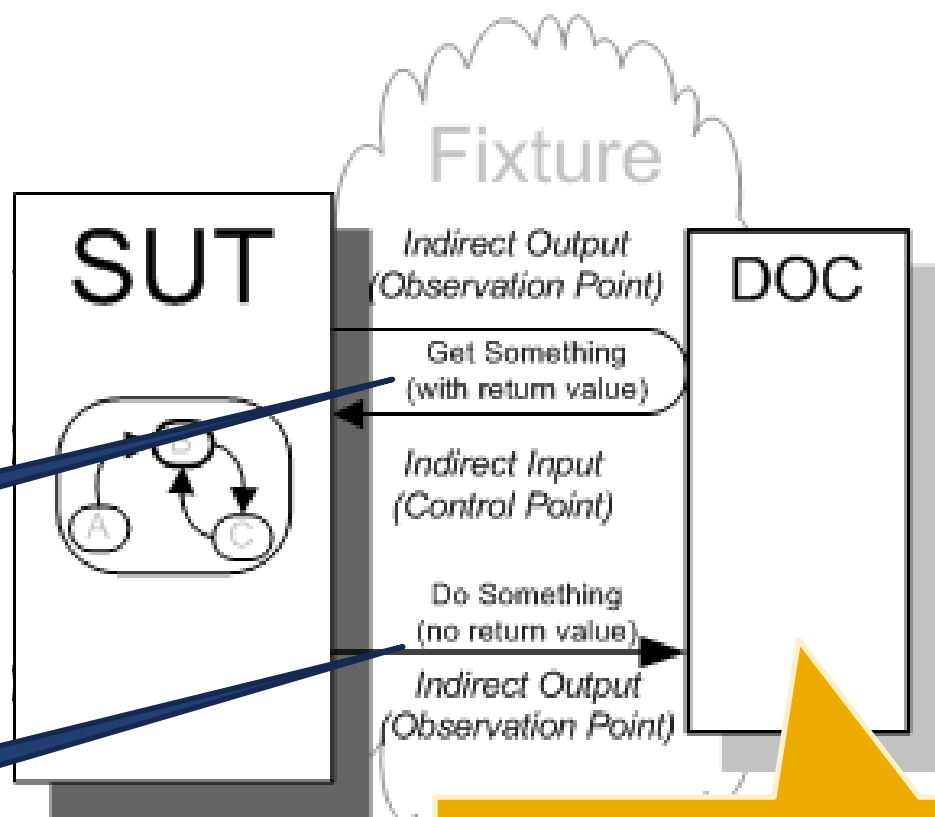
Andamiaaje

Esquema de test de integración



Andamiaaje

- ❑ En pruebas de integración, puede interesar comprobar cómo se produce la interacción entre SUT y DOC.



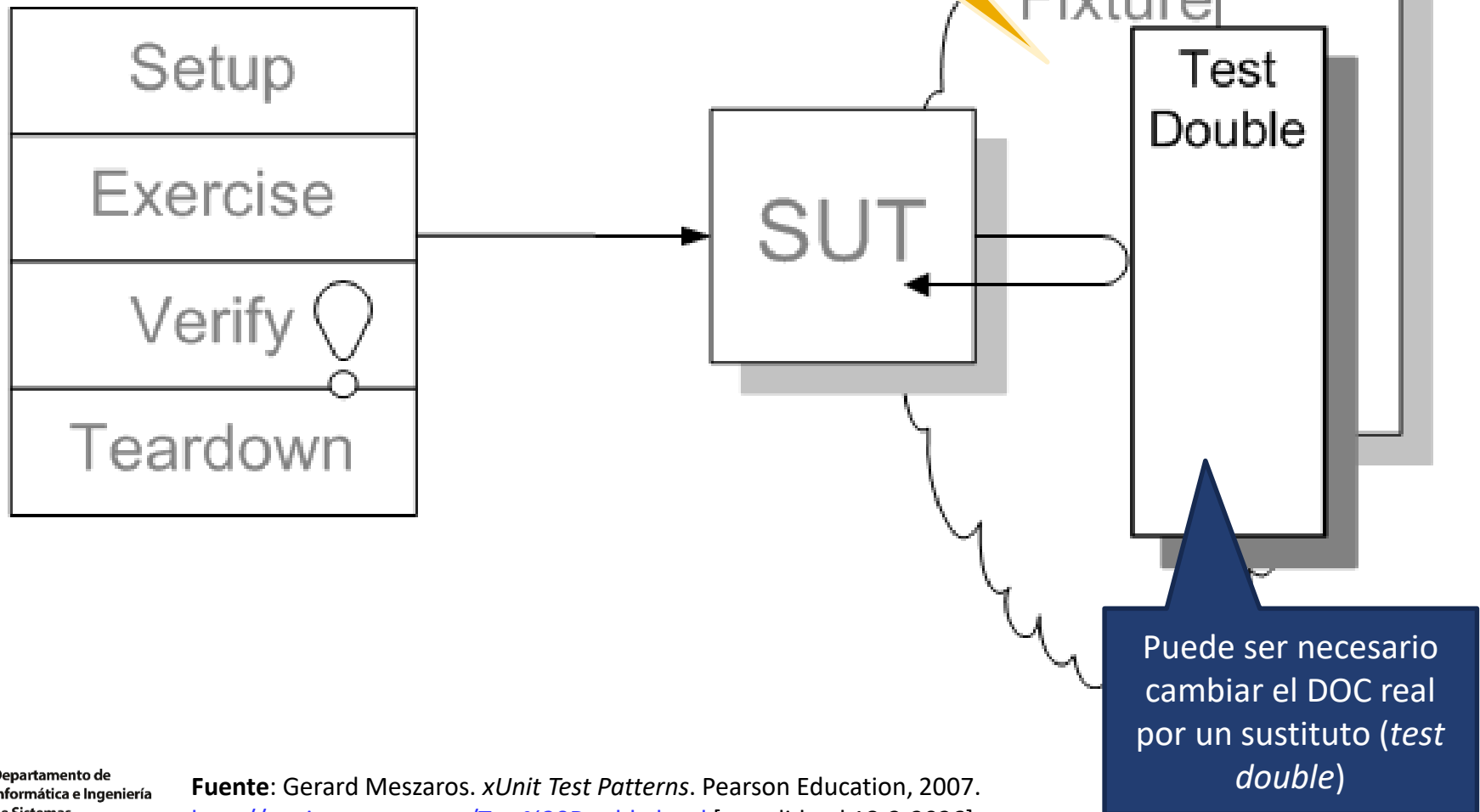
Esta interacción se puede comprobar indirectamente

Esta interacción no

- También puede que el DOC:
- no esté implementado
 - sea muy lento
 - sea costoso de configurar
 - ...

Andamiaaje

Puede ser necesario realizar una configuración especial para probar las interacciones entre SUT y DOC



Andamiaje

Terminología de Gerard Meszaros

- ❑ *Dummy*
 - ❖ Objetos que el objeto de pruebas **no va a utilizar** en la prueba.
- ❑ ***Stub***
 - ❖ Proporcionan los resultados adecuados para una prueba concreta. **Solo es relevante la información que el *stub* proporciona al objeto de pruebas.**
- ❑ *Spy*
 - ❖ Registran las salidas indirectas (salidas del objeto de pruebas al colaborador). **La información relevante es la que el *spy* recibe del objeto de pruebas**, en particular, el estado que alcanza tras la realización de las pruebas.
- ❑ ***Mock***
 - ❖ Sustitutos con una especificación del comportamiento que tienen que tener y de cómo van a ser invocados en la prueba. **Son relevantes las entradas y las salidas al colaborador, así como las invocaciones de la unidad objeto de pruebas al colaborador (comportamiento).**
- ❑ *Fake*
 - ❖ Implementaciones ligeras pero totalmente funcionales del colaborador real. No son adecuados para producción. El objeto de pruebas va a **interactuar** con ellos.
- ❑ *Proxy*
 - ❖ **Objeto para pruebas interpuesto** entre un colaborador real y la unidad objeto de pruebas en el contexto de una prueba.

Dummy

- ❑ Objetos (parámetros, atributos) que el objeto de pruebas no va a utilizar en la prueba en concreto.
- ❑ Se trata de objetos que en la ejecución real van a existir y tener un estado concreto (y a veces son complicados de generar), pero que en una prueba concreta no se van a utilizar.
- ❑ Cuando aparecen como parámetro de un método del SUT o son atributos del SUT, es necesario pasar un argumento al parámetro, o inicializar el atributo.
- ❑ Utilizar un *dummy* para el parámetro o atributo permite ahorrarse la generación e inicialización del objeto real.

Ejemplo de *dummy*

- ❑ En las pruebas de sistema de la práctica 3 hay que llamar al método `Main.main(String[] args)`.
- ❑ El parámetro `args` representa la invocación realizada desde la línea de comandos.
- ❑ El método `Main.main` no lo utiliza nunca.
- ❑ En lugar de generar un vector de `String`, es posible invocar al método `main` con el valor `null`.

Ejemplo de *dummy*

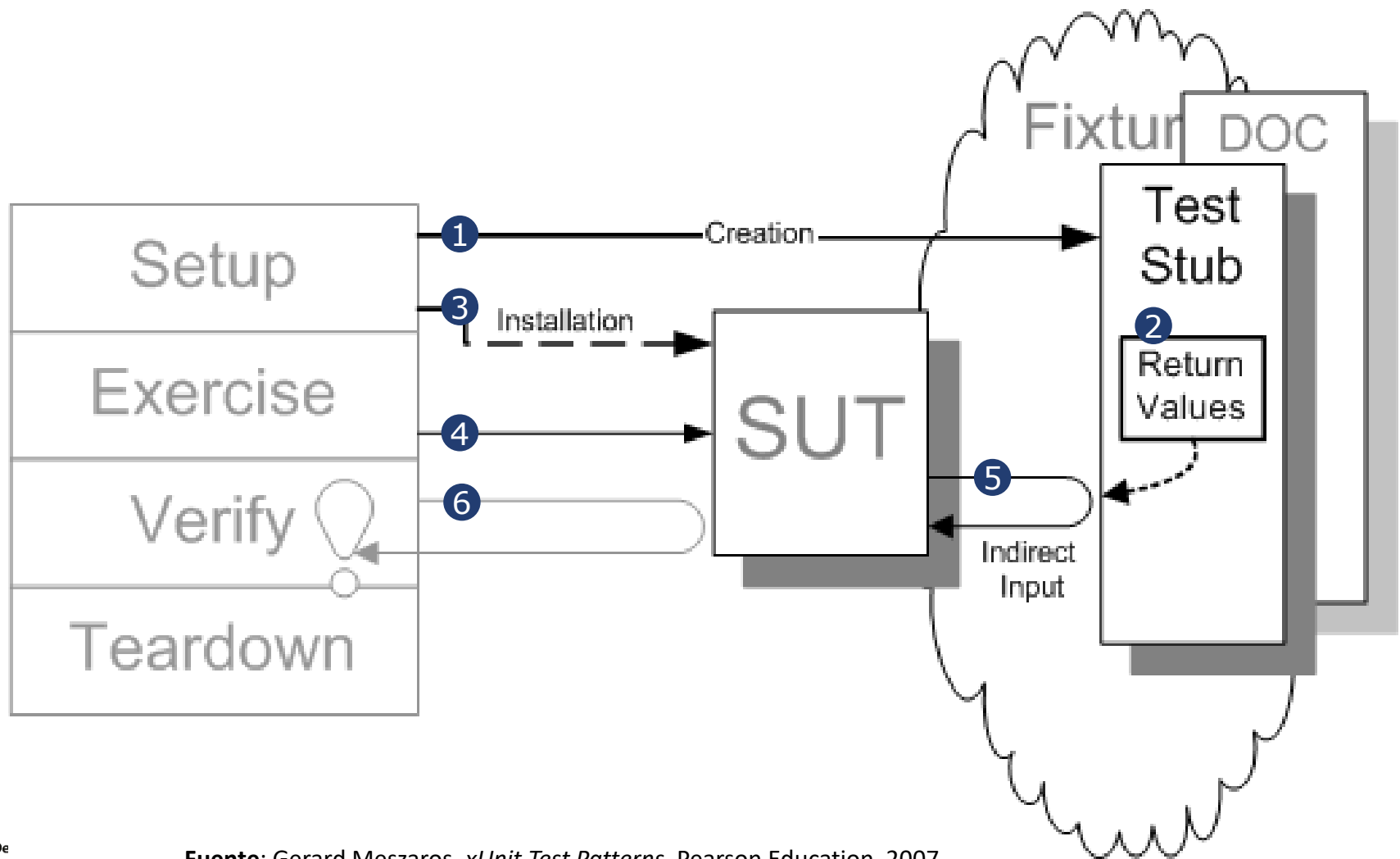
```
public class MainTest {  
    @Test  
    public void testQuijote() {  
        // Modificación de entrada y salida estándares  
        ...  
  
        // Ejecución del SUT  
        Main.main(null);  
  
        // Comprobación de resultados  
        ...  
    }  
}
```

Se invoca con `null` en lugar de `new String[]{} o new String[]{"arg0", "arg1", "arg2"}`

Stubs

- ❑ Objetos *ad hoc* para una prueba que proporcionan los resultados adecuados para esa prueba concreta. **Solo es relevante la información que el *stub* proporciona a la unidad.**
- ❑ Uso de un *stub*
 1. El test crea y configura el *stub*.
 2. Si es necesario, el test configura el SUT para que utilice el *stub*.
 3. El test se ejecuta, haciendo que la unidad utilice el *stub*.
 4. El SUT interactúa con el *stub*.
 5. El *stub* devuelve los valores al SUT con los que fue configurado.
 6. El test comprueba el estado del SUT.

Stubs



Ejemplo de *stub*

```
public class StubStream extends InputStream
    implements DataInput {

    @Override
    public int readInt() throws IOException {
        return 8;
    }

    ...
}
```

Este *stub* sustituiría a un fichero real. El contenido del fichero sería una secuencia infinita de ochos.

Ejemplo de *stub*

```
public class SaboteurStream
    extends InputStream implements DataInput {

    @Override
    public int readInt() throws IOException {
        throw new IOException("Hardware error");
    }

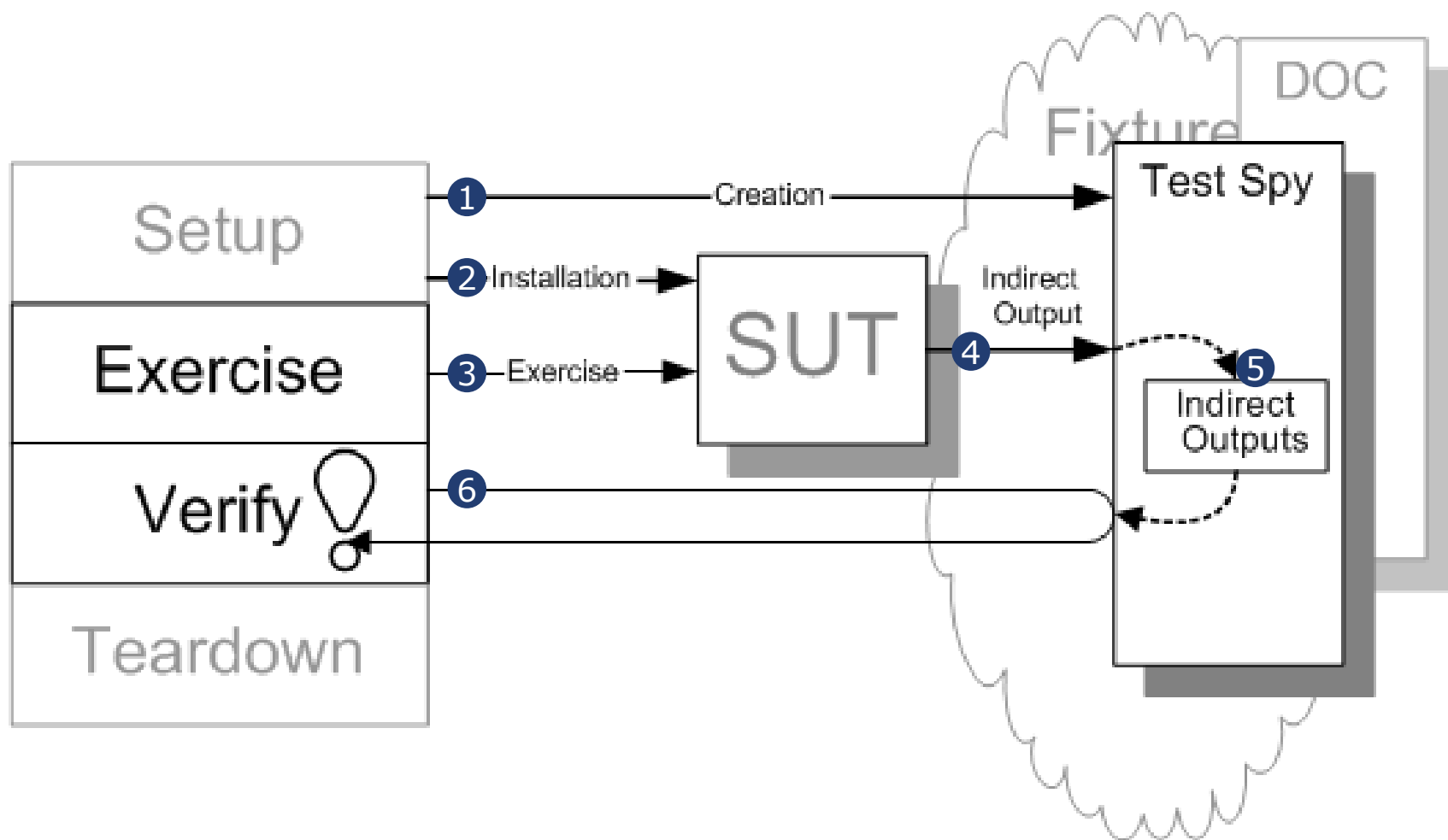
    ...
}
```

Este *stub* sustituiría a un fichero real, simulando un error de *hardware*.

Spy

- ❑ Objetos *ad hoc* para una prueba que registran las salidas indirectas (salidas del objeto de pruebas al colaborador). La información más relevante es la que el *spy* recibe de la unidad, en particular, el **estado** que alcanza tras la realización de las pruebas.
- ❑ Uso de un *spy*:
 1. El test crea o configura el *spy*.
 2. El test configura el SUT para que use el *spy*.
 3. El test se ejecuta.
 4. El SUT interactúa con el *spy*.
 5. El *spy* almacena los datos de la interacción.
 6. El test comprueba el estado del ***spy***, no del SUT.

Spy



Mocks

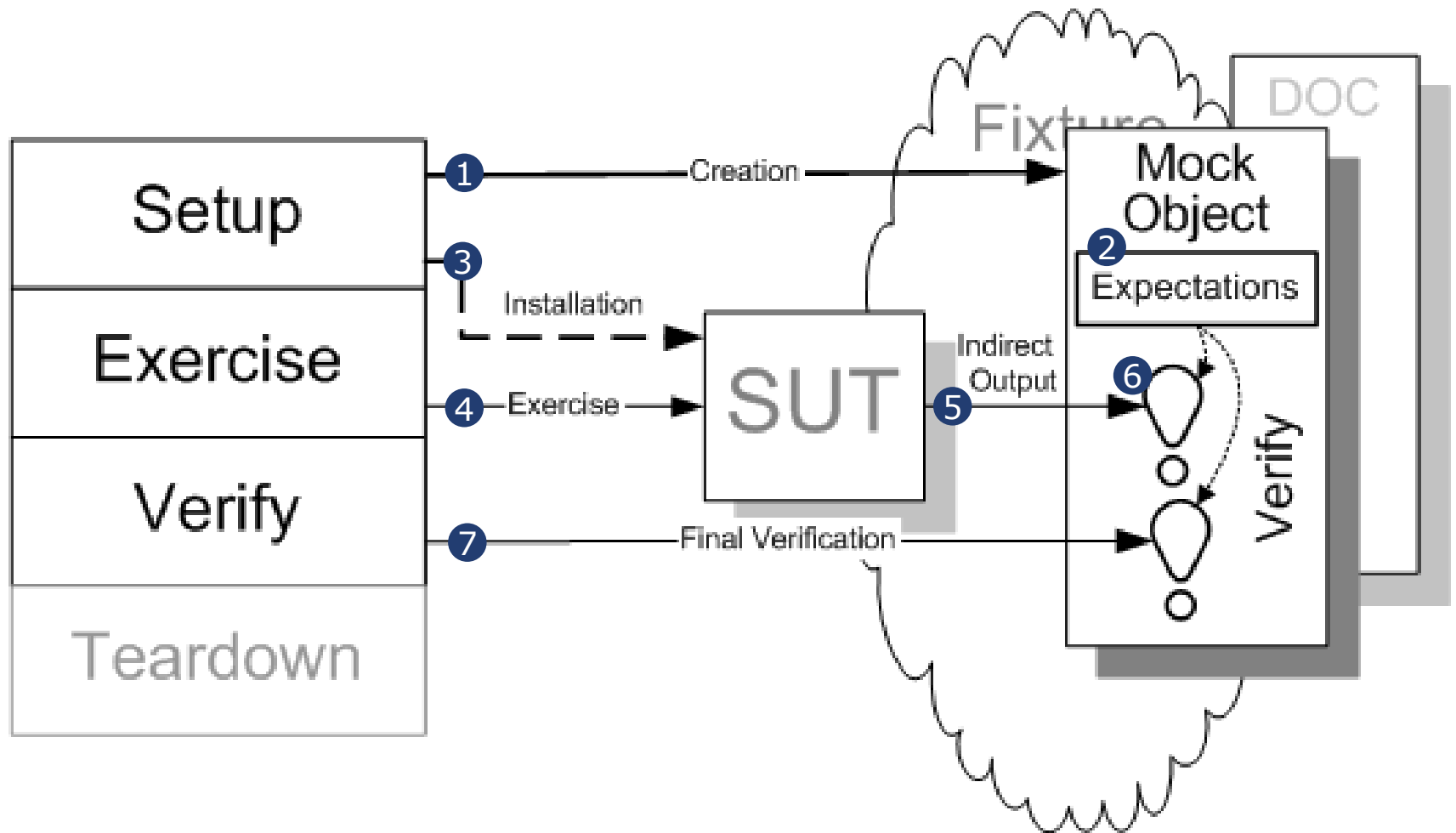
- Objetos para pruebas con una especificación del comportamiento que tienen que tener y de cómo van a ser invocados en la prueba. Son relevantes tanto las entradas como las salidas al colaborador, así como las invocaciones de la unidad objeto de pruebas al colaborador (**comportamiento**).

Mocks

□ Uso de un *mock*:

1. El test crea el *mock*.
2. El test configura el *mock* estableciendo los resultados de interacción esperados (*expectations*).
3. Si es necesario, el test configura el SUT para que utilice el *mock*.
4. El test se ejecuta.
5. El SUT interactúa con el *mock*.
6. El *mock* almacena los datos de la interacción.
7. El *mock* comprueba que se ha producido la interacción esperada, validando las *expectations*.

Mocks



Fakes

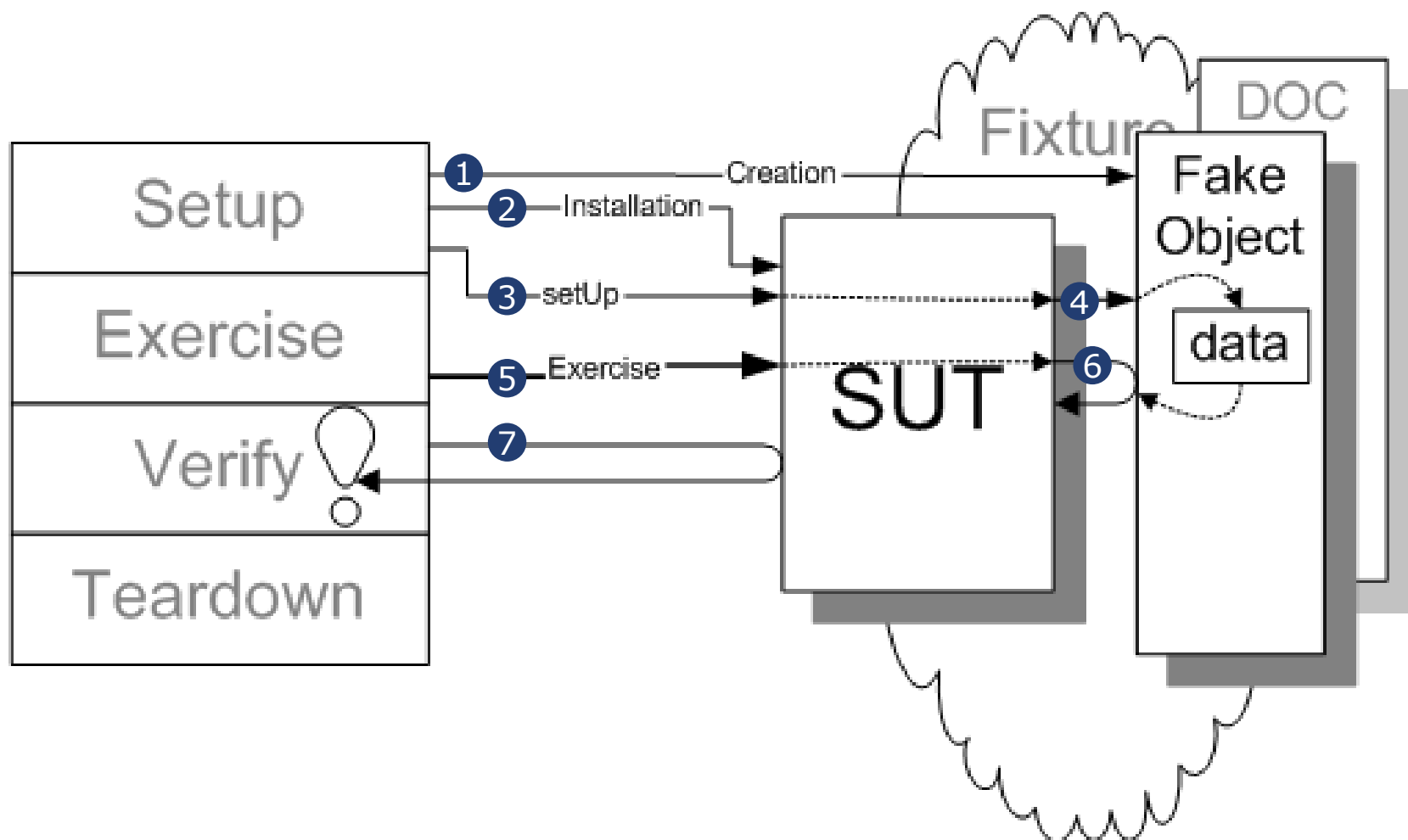
- ❑ Objetos que ofrecen la misma interfaz que el colaborador real, pero con una implementación más ligera y una puesta en marcha más sencilla.
- ❑ El test no interactúa con el *fake*.
- ❑ Pueden no ser adecuados para producción (carencia de escalabilidad, persistencia, ...).
- ❑ Ejemplos de *fakes*
 - ❖ Base de datos implementada con tablas *hash*
 - ❖ Base de datos en memoria
 - ❖ Servidor web ligero

Fakes

□ Uso de un *fake*

1. El test crea el *fake*.
2. El test crea el SUT y lo configura para que utilice el *fake*.
3. El test configura el *fake*
4. En los casos más complicados, lo configura a través del SUT
5. El test se ejecuta, haciendo que el SUT use el *fake*.
6. El SUT interactúa con el *fake*.
7. El test comprueba el estado del SUT.

Fakes



Proxies

- ❑ Objeto para pruebas interpuesto entre un colaborador real y la unidad objeto de pruebas en el contexto de una prueba.
- ❑ Uso de un *proxy*:
 - ❖ El test crea y configura el colaborador.
 - ❖ El test crea y configura el *proxy* para que use el colaborador real.
 - ❖ El test se ejecuta, haciendo que la unidad utilice el *proxy*.
 - ❖ El test comprueba el estado de la unidad.
 - ❖ El test comprueba el estado del colaborador real utilizando el *proxy*.

Proxy

```
public class ProxyStream extends InputStream implements DataInput {

    private DataInputStream realStream;
    private int numberOfTimesClosed = 0;

    public ProxyStream(DataInputStream realStream) {
        this.realStream = realStream;
    }

    @Override
    public void close() throws IOException {
        super.close();
        realStream.close();
        numberOfTimesClosed++;
    }

    @Override
    public int readInt() throws IOException {
        return realStream.readInt();
    }

    ...
}
```

Ejercicio

- En el contexto de la práctica 3,
¿qué sería (*dummy*, *stub*, *spy*, *mock*, *fake*, *proxy*)?
 - ❖ Utilizar `ByteArrayInputStream` para simular la lectura de un fichero a través de un `FileInputStream` o de la entrada estándar.
 - ❖ Utilizar `ByteArrayOutputStream` para simular la escritura en un fichero a través de `FileOutputStream` o de la salida estándar.

Andamiaje

Diferencias principales

<i>Dummy</i>	El DOC no importa.
<i>Stub</i>	El DOC tiene que proporcionar datos concretos al SUT.
<i>Spy</i>	Se quiere comprobar que el SUT interactúa de una determinada forma con el DOC y que proporciona datos concretos al DOC. El test comprueba el estado del <i>spy</i> .
<i>Mock</i>	Se quiere comprobar que el SUT interactúa de una determinada forma con el DOC y que proporciona datos concretos al DOC. El test delega en el <i>mock</i> la comprobación de la interacción
<i>Fake</i>	Se va a interactuar con el DOC, que se sustituye por una implementación ligera. No interesa la interacción en concreto.
<i>Proxy</i>	Como el <i>mock</i> , pero interpuesto entre el SUT y el DOC real .

Configuración de pruebas con dependencias

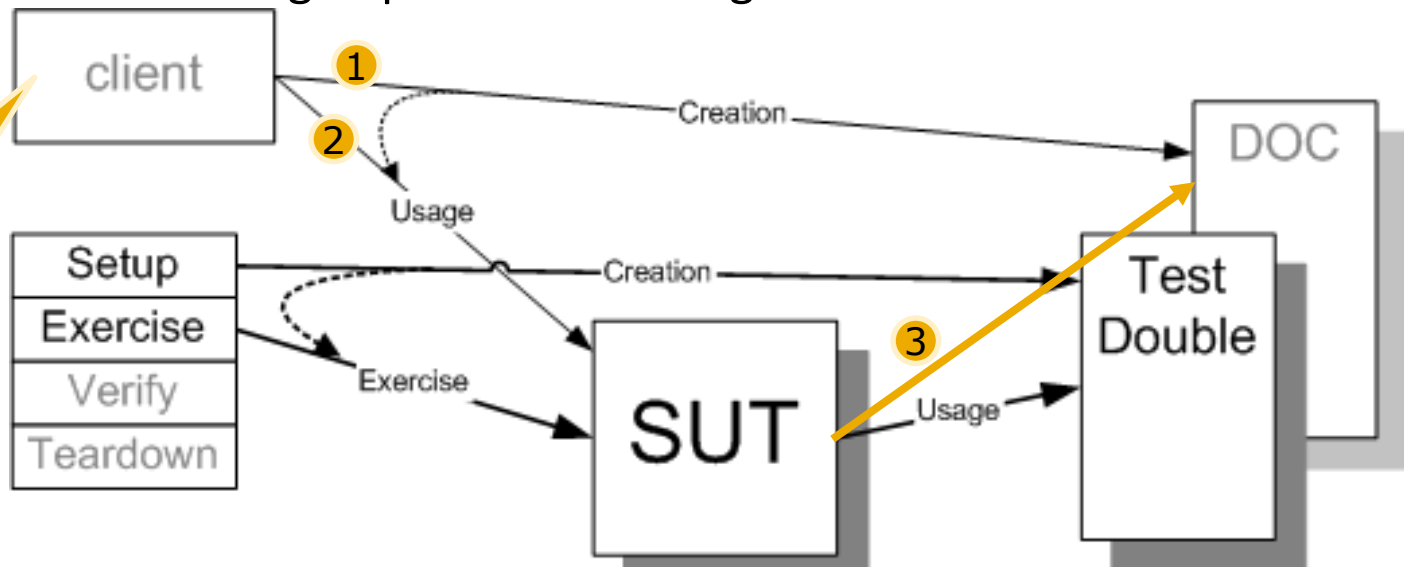
- ❑ Cómo diseñar la unidad objeto de pruebas (SUT) para poder modificar sus dependencias (COD) en tiempo de ejecución.
- ❑ Estrategias principales
 - ❖ Inyección de dependencias
 - ❖ Búsqueda de dependencias
 - ❖ Enganches
 - ❖ *Humble object o Window object o Page object*
 - Estrategia más dirigida a interfaces con el usuario u otro código difícil de probar
 - Consiste en proporcionar una API para los elementos de una interfaz de usuario contra la que hacer las pruebas.

Inyección de dependencias

❑ *Dependency injection*

❑ El cliente (sistema en producción o *test*) crea la dependencia (COD o sustituto) y se la pasa al SUT

- ❖ En el constructor
- ❖ A través de algún método modificador (*setter*)
- ❖ A través de algún parámetro de algún método



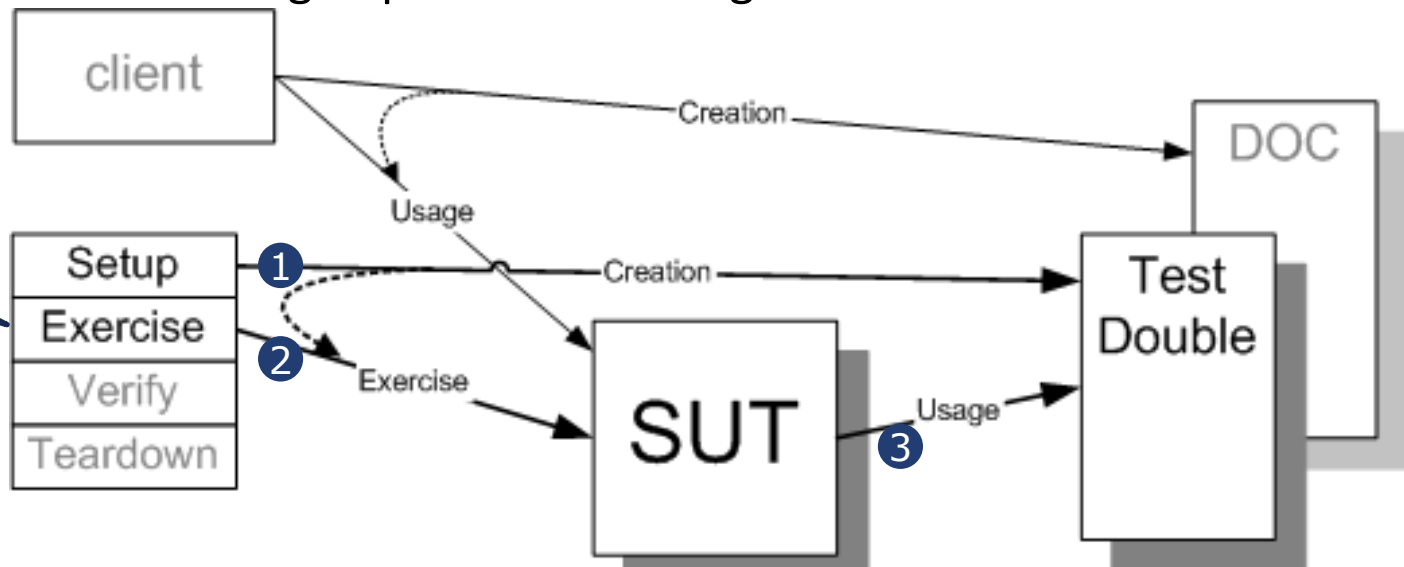
En tiempo
de ejecución

Inyección de dependencias

❑ *Dependency injection*

❑ El cliente (sistema en producción o *test*) crea la dependencia (COD o doble) y se la pasa al SUT

- ❖ En el constructor
- ❖ A través de algún método modificador (*setter*)
- ❖ A través de algún parámetro de algún método

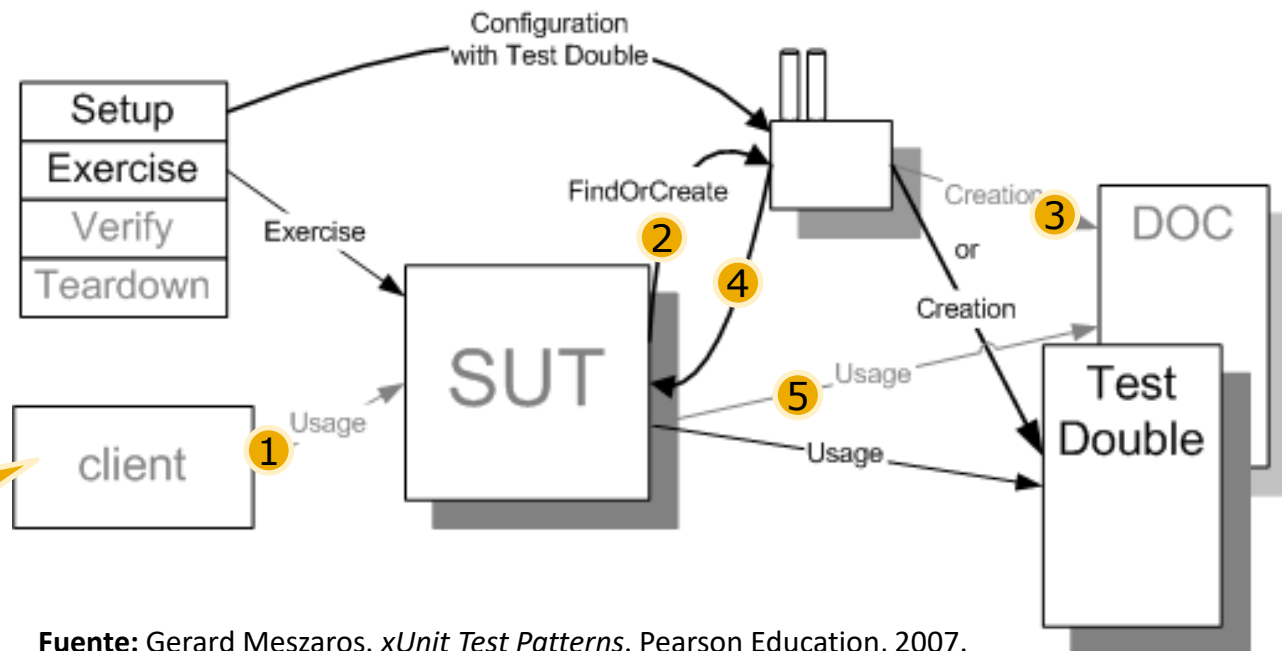


Búsqueda de dependencias

❑ *Dependency Lookup*

❑ La unidad testada (SUT) obtiene la dependencia de algún objeto antes de utilizarla.

- ❖ En producción, ese objeto devuelve la dependencia real (COD)
- ❖ En *testing*, devuelve un doble



En tiempo de ejecución

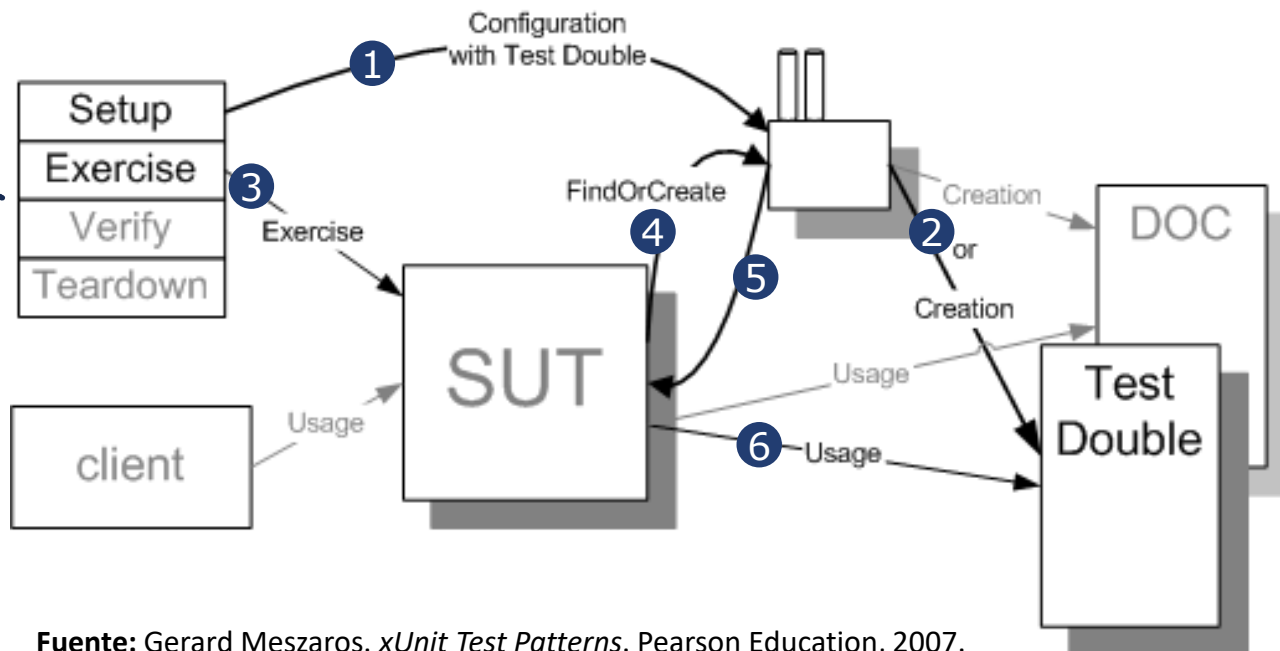
Búsqueda de dependencias

❑ *Dependency Lookup*

❑ La unidad testada (SUT) obtiene la dependencia de algún objeto antes de utilizarla.

- ❖ En producción, ese objeto devuelve la dependencia real (COD)
- ❖ En *testing*, devuelve un doble

En tiempo
de *testing*



Enganches

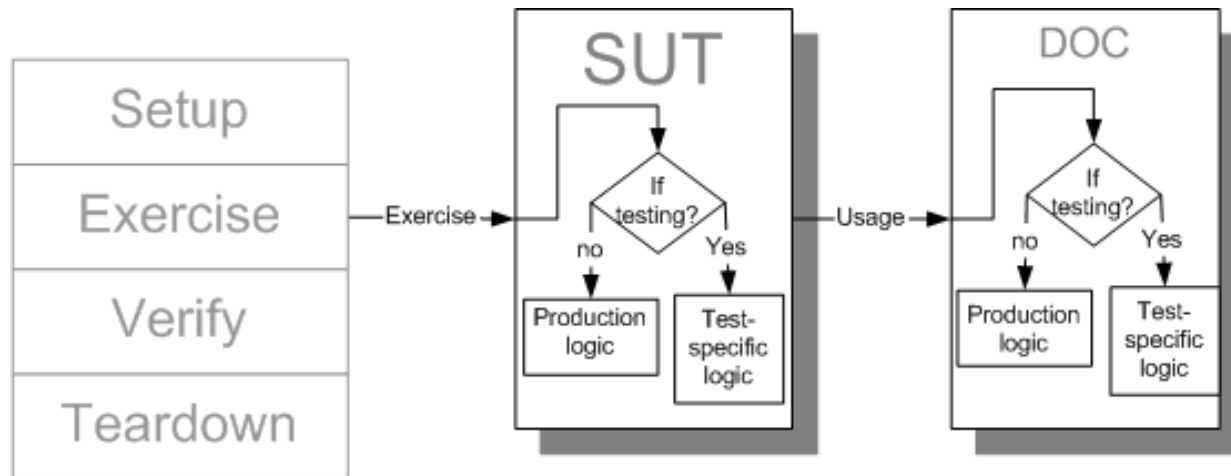
❑ *Test hook*

❑ Se modifica el sujeto de pruebas (o incluso la dependencia) para que se comporte de forma diferente durante la prueba.

❖ Se necesitan uno o más *flags*.

❑ Estrategia de último recurso

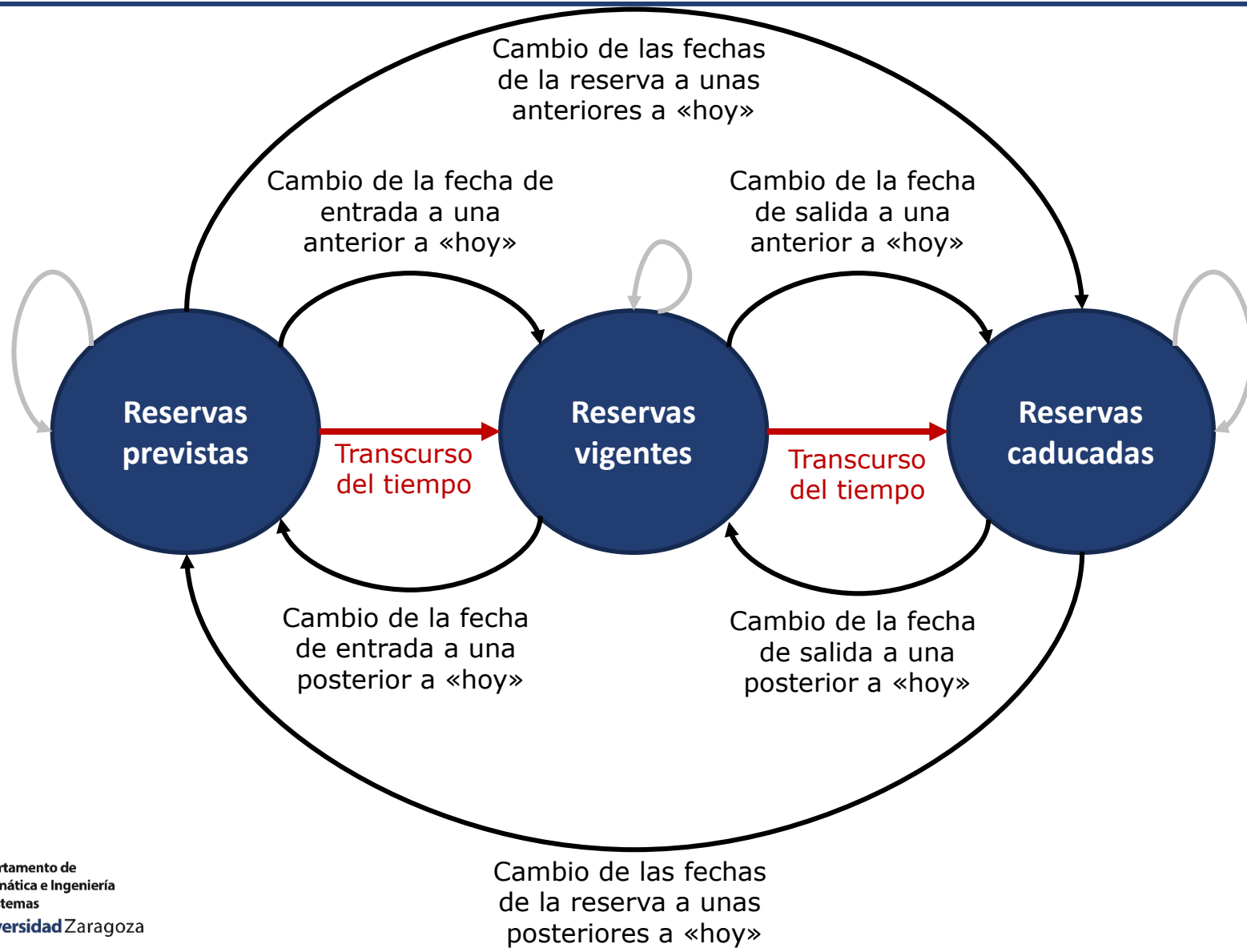
❖ Son preferibles la inyección o búsqueda de dependencias



En el trabajo...

- ❑ Las reservas creadas y no borradas pueden estar en uno de estos tres estados:
 - ❖ Previstas
 - ❖ Vigentes
 - ❖ Caducadas
- ❑ ¿Qué transiciones hay entre ellas?
- ❑ ¿Hay alguna que pueda suponer un problema para hacer pruebas?

En el trabajo...



En el trabajo...

- ❑ Prueba de desarrollo o de sistema sobre el estado de vigencia de los pedidos. Dicha prueba será diseñada utilizando la técnica de caminos con profundidad 1.
- ❑ Va a ser necesario gestionar la dependencia con la fecha del sistema:
 - ❖ Inyección de la fecha
 - ❖ Búsqueda de la fecha

Bastará con cubrir las aristas negras y rojas del grafo.

Tutoría

Estudiar la mejor estrategia de gestión de dependencias

Andamiaje. Herramientas



<https://mockito.org/>

EASYMOCK

<https://www.easymock.org/>



<http://jmock.org>



<https://jmockit.github.io/>

Ejemplos

- ❑ Tenemos una clase (MiDependencia) que va a ser una dependencia de cierto objeto de pruebas (SUT)

MiDependencia
+obtenerId(): int +probar(i: int) +noUsar() +llamar5veces()

- ❑ En los test que siguen, el objeto de pruebas va a ser indiferente. Para mostrar la funcionalidad de Mockito, los test van a interactuar solo con el *mock*, no con un objeto de pruebas.

Mockito. *Stubbing*

```
@Test
public void testSetReturnValue_1() {
    // create stub
    MiDependencia stub =
        Mockito.mock(MiDependencia.class);

    // define return value for method obtenerId()
    when(stub.obtenerId()).thenReturn(43);

    // use mock in test...
    assertEquals(43, stub.obtenerId());
}
```



Mockito. Stubbing

```
@Test
public void testSetReturnValue_2() {
    // create stub
    MiDependencia stub =
        Mockito.mock(MiDependencia.class);

    // define return value for method obtenerId()
    doReturn(43).when(stub).obtenerId();

    // use mock in test...
    assertEquals(43, stub.obtenerId());
}
```



Mockito. *Stubbing*

```
// Demonstrates the return of multiple values
@Test
public void testMoreThanOneReturnValue_1() {
    Iterator iterator = mock(Iterator.class);
    when(iterator.next()).thenReturn("Mockito")
                               .thenReturn("rocks");
    String result = iterator.next() + " "
                       + iterator.next();
    assertEquals("Mockito rocks", result);
}
```



«interface»
Iterator

+hasNext(): boolean
+next(): E
...

E

Mockito. *Stubbing*

```
// Demonstrates the return of multiple values
@Test
public void testMoreThanOneReturnValue_2() {
    Iterator iterator = mock(Iterator.class);
    when(iterator.next()).thenReturn("Mockito",
                                    "rocks");

    String result = iterator.next() + " "
                    + iterator.next();
    assertEquals("Mockito rocks", result);
}
```



«interface»
Iterator

+hasNext(): boolean
+next(): E
...

E

Mockito. *Stubbing*

```
// Demonstrates how to return values based on the input
@Test
public void testReturnValueDependentOnMethodParameter(){
    Comparable c = mock(Comparable.class);
    when(c.compareTo("Mockito")).thenReturn(1);
    when(c.compareTo("Eclipse")).thenReturn(2);
    assertEquals(1, c.compareTo("Mockito"));
}
```

T

«interface»

Comparable

+compareTo(obj: T): int



Mockito. *Stubbing*

```
@Test
public void
testReturnValueIndependentOnMethodParameter() {
    Comparable c = mock(Comparable.class);
    when(c.compareTo(anyInt())).thenReturn(-1);
    assertEquals(-1, c.compareTo(9));
    assertEquals(0, c.compareTo("Mockito"));
}
```

T

«interface»

Comparable

+compareTo(obj: T): int



Mockito. *Stubbing*

```
// This test demonstrates how use doThrow

@Test(expected = IOException.class)
public void testForIOException() throws IOException {

    // create and configure stub
    OutputStream stubStream = mock(OutputStream.class);
    doThrow(new IOException()).when(stubStream).close();

    // use stub
    OutputStreamWriter streamWriter
        = new OutputStreamWriter(stubStream);
    streamWriter.close();
}
```

Fuente: Lars Vogel, Fabian Pfaff. «Unit tests with Mockito – Tutorial». *Vogella*. Version 2.3. 16-8-2021.

<https://www.vogella.com/tutorials/Mockito/article.html>

[accedido el 19-3-2024]

OutputStream
+close(): void {exceptions=IOException}
...

```
+obtenerId(): int  
+probar(i: int)  
+noUsar()  
+llamar5veces()
```

Mockito. Mocking

```
@Test  
public void testVerify() {  
    // Creación y configuración del mock  
    MiDependencia mock =  
        Mockito.mock(MiDependencia.class);  
  
    // Uso del mock  
    mock.probar(12);  
    mock.obtenerId();  
    mock.obtenerId();  
    for (int i = 0; i < 5; i++) {  
        mock.llamar5veces();  
    }  
  
    ...  
}
```

El uso del *mock* normalmente lo haría el objeto de pruebas, no el test



Mockito. Mocking

```
@Test
public void testVerify() {
    ...

    // Comprobación: el método «probar»
    // ha sido invocado una vez con el parámetro 12
    verify(mock).probar(12);

    // Comprobación: el método obtenerId ha sido
    // invocado 2 veces
    verify(mock, times(2)).obtenerId();

    ...
}
```



Mockito. Mocking

...

```
// el método «noUsar» no se invocó nunca  
verify(mock, never()).noUsar();
```

```
// el método «llamar5veces» se invocó 5 veces  
// exactas  
verify(mock, times(5)).llamar5veces();
```

...



Mockito. Mocking

...

```
// «probar» con parámetro 12 se invocó
// al menos una vez
verify(mock, atLeastOnce()).probar(12);

// el método «obtenerId» se invocó al menos 2 veces
verify(mock, atLeast(2)).obtenerId();

// el método «obtenerId» se invocó como mucho
// 3 veces
verify(mock, atMost(3)).obtenerId();

// comprobar que no ha habido otras interacciones con
// el mock
verifyNoMoreInteractions(mock);
}
```



Mockito. Terminología

- ❑ Lo que hemos llamado *mocking* con Mockito se realizaría con un *spy* según la terminología de Gerard Meszaros.
 - ❖ Las verificaciones de las interacciones las hace el test y no el propio sustituto.
- ❑ En EasyMock, JMock y JMockit sí que hay *mocks* estrictos.

Mockito. Más funcionalidad

❑ *Proxies*

- ❖ `List list = new LinkedList();`
- ❖ `List spy = Mockito.spy(list);`

¡Atención! Un *spy* en Mockito es un *proxy* según la terminología de Meszaros.

❑ Inyección de dependencias

❑ Sintaxis alternativa basada en anotaciones

- ❖ `@Mock private MiDependencia mock;`
- ❖ `@Spy private List spy = new LinkedList();`
- ❖ `@Rule public MockitoRule mockitoRule = MockitoJUnit.rule();`



Mockito

□ Dos ejercicios

❖ Patrón sujeto-observador

➤ Papeles

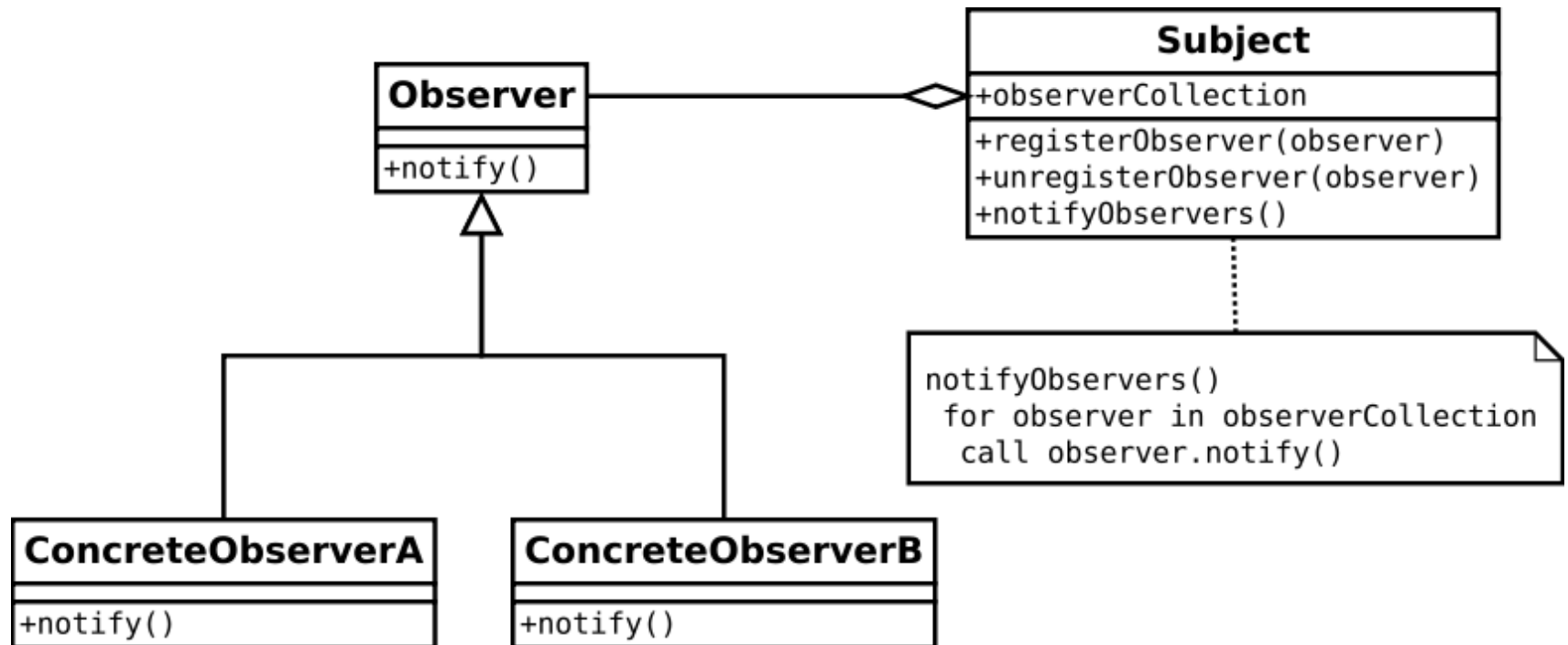
- Sujeto: Objeto de pruebas (SUT)
- Observador: Colaborador (DOC)

- Pruebas de unidad del sujeto sin implementar observadores
- Uso de Mockito con *mocks* para verificar la interacción correcta con los observadores

❖ Análisis de un fichero

- Pruebas de unidad de un método sin utilizar ficheros reales
- Uso de Mockito con *stubs*.
- Inyección de errores para alcanzar el 100 % de cobertura

Ejemplo: Patrón Sujeto-Observador



Problema: Patrón Sujeto-Observador

❏ Repositorio IntelliJ en GitHub

❖ <https://github.com/unizar-vv-organization/unizar-vv-mockito>

❖ Paquete `es.unizar.eina.vv.mockito.ejercicio1`

- Interfaz *observadora* denominada `Subscriber` con un método `recieve(String)`
- Interfaz *sujeto* denominada `Publisher` con los métodos `add(Subscriber)` y `publish(String)`
- Clase `MyPublisher` que implementará la interfaz `Publisher`
 - No la implementéis antes de escribir los test

Mockito

❏ Configuración del proyecto en IntelliJ y Android Studio

❖ *Project Structure | Libraries | + (New Project Library) | From Maven...*

➤ org.junit.jupiter:junit-jupiter:6.0.1

➤ org.mockito:mockito-junit-jupiter:5.23.0

❏ Más información:

❖ <https://site.mockito.org/>

❖ <https://www.javadoc.io/doc/org.mockito/mockito-core/latest/index.html>

Problema: Patrón Sujeto-Observador

- ❑ Escribid una prueba JUnit para la clase `MyPublisher`:
 - ❖ Cread un objeto *mock* de la interfaz `Subscriber`.
 - ❖ Suscribidlo a un objeto de la clase `MyPublisher`.
 - ❖ Publicad un mensaje en el objeto de la clase `Publisher`.
 - ❖ Comprobad sobre el *mock* si la interacción ha sido correcta.
- ❑ Implementad y corregid la clase `MyPublisher` hasta que el test no detecte fallos.
- ❑ Añadid pruebas adicionales que cubran los siguientes escenarios:
 - ❖ Dos *mocks* y un mensaje a ambos:
 - Dos *mocks* de la interfaz `Subscriber` se suscriben al `Publisher`.
 - El `Publisher` publica un mensaje.
 - Comprobad que los dos *mocks* reciben el mensaje.
 - ❖ Dos *mocks* y dos mensajes distintos a ambos
 - ❖ Dos *mocks* y dos mensajes, pero entre la suscripción del primer y el segundo *mock*, el `Publisher` publica el primer mensaje.

Ejercicio con Mockito.

Sustitución de la clase DataInputStream

Nombre del fichero de ventas: v2014.dat

Resumen de ventas por cliente almacenados en el fichero anterior

Cliente	Importe
-----	-----
150	15,00
159	250,00

Ejercicio con Mockito.

Sustitución de la clase DataInputStream

- ❑ La estructura de los ficheros binarios de ventas responde al siguiente esquema:
 - ❖ `<fichero_ventas> ::= { <venta> }`
 - ❖ `<venta> ::=`
 - `<producto> <cliente> <cantidad> <precioUnitario>`
 - ❖ `<producto> ::= int` *-- Código de producto*
 - ❖ `<cliente> ::= int` *-- Código de cliente*
 - ❖ `<cantidad> ::= int` *-- Unidades vendidas*
 - ❖ `<precioUnitario> ::= double` *-- Precio unitario*
- ❑ Los datos de ventas de estos ficheros están ordenados en orden creciente de códigos de cliente, de forma que las ventas de un mismo cliente están almacenadas, necesariamente, de forma consecutiva.

Problema

Venta

–producto: int
–cliente: int
–cantidad: int
–precioUnitario: double

«constructor»

+Venta(producto: int, cliente: int, cantidad: int, precioUnitario: double)

+producto(): int

+cantidad(): int

+precioUnitario(): double

+cliente(): int

+leerVenta(DataInputStream f): Venta {exceptions=IOException}

Ejercicio con Mockito.

Sustitución de la clase DataInputStream

```
/**  
 * Pre: f es no nulo y permite leer de un fichero  
 *      binario de ventas con la estructura y  
 *      características planteadas en el enunciado.  
 * Post: Ha escrito en la pantalla un resumen de las  
 *       ventas, listando, para cada código de cliente,  
 *       el importe total de las compras que ha  
 *       realizado.  
 */  
protected static void escribirResumen(DataInput f) {  
    ...  
}
```

Ejercicio con Mockito.

Sustitución de la clase `DataInputStream`

- Al aplicar la técnica de pruebas de caminos con niveles de profundidad 2, se obtuvieron 4 caminos, que se correspondían con ficheros con las siguientes características.
 - ❖ Fichero vacío (con 0 ventas)
 - ❖ Fichero con solo una venta
 - ❖ Fichero con dos ventas de dos clientes distintos
 - ❖ Fichero con 4 ventas de dos clientes distintos c_1 y c_2 . Las dos primeras son del cliente c_1 y las dos últimas, del cliente c_2 .

Ejercicio con Mockito.

Sustitución de la clase `DataInputStream`. Tareas

1. JUnit para probar el método `escribirResumen`, recorriendo los cuatro caminos identificados
 - ❑ En lugar de ficheros binarios reales, utilizad Mockito para crear objetos sustitutos (*stubs*, en concreto) de la interfaz `DataInput`
 - ❑ Solo configurad las pruebas y ejecutarlas
No comprobéis de momento los resultados de la ejecución (es decir, no redirijáis de momento la salida estándar a ningún otro sitio)

¡Atención! No es recomendable hacer *mocks* de clases que no nos pertenezcan.
¿Por qué?

<https://github.com/mockito/mockito/wiki/How-to-write-good-tests#dont-mock-a-type-you-dont-own>

Ejercicio con Mockito.

Sustitución de la clase `DataInputStream`. Tareas

2. Comprobad en IntelliJ si se alcanza una cobertura del 100 % en el método `escribirResumen`.
3. Utilizad Mockito para añadir una prueba en la que **inyectar el error** necesario para alcanzar una cobertura del 100 %.
4. Terminad de automatizar las pruebas, comprobando los resultados de la ejecución de las mismas (redirigiendo la salida estándar a un *mock*, que puede ser un `ByteArrayOutputStream`).

